



Course Code: EEE373
Course Title: Embedded System Design

Prepared by:

Afrida Islam
ID: 21221030
Section: 02

Mid Assignment

Answer to the question no. 1

(a) The resistor connected between a microcontroller input pin and the positive voltage supply (V_{CC}) with a switch button, is known as a pull-up resistor. Pull-up resistors are necessary for inputting data into a microcontroller to ensure stable and consistent readings. When the input pin is not driven by any external device, the resistor ensures that the input pin reads a specific logic high level (1). This prevents the pin from floating, which can cause unpredictable behaviour due to noise or interference.

In order to connect internal pull-up resistors to an ATmega32, first we need to set the corresponding bit in the PORT register to activate the pull-up resistor and then we need to clear the corresponding bit in the Data Direction Register (DDR) to configure the pin as an input.

(b)

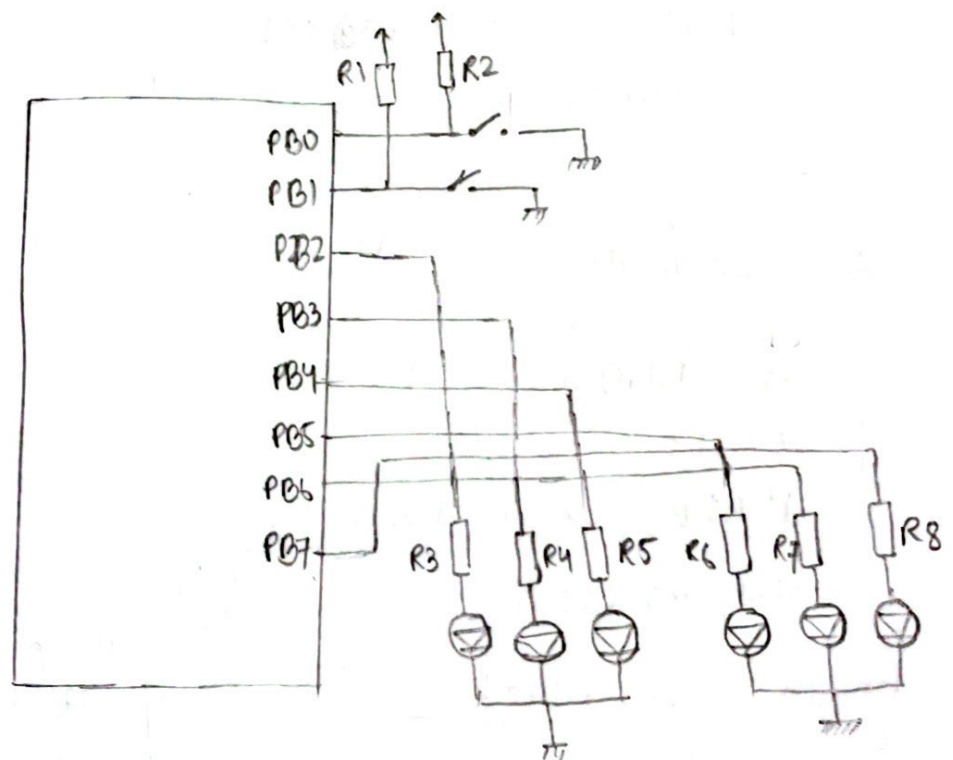


fig: Circuit diagram of the given system

```
(c) #include <mega32.h>
```

```
#include <delay.h>
```

```
void main (void)
```

```
{
```

```
    DD RB = 0b11111100; // PB0 and PB1 are inputs and PB2 to PB7 are  
                        LED outputs.
```

```
    while (1)
```

```
    {
```

```
        // Left turn switch connected to PB0
```

```
        if ( ( PINB & (1<< PB0 PINB0) ) == 0 ) // Active low on PB0
```

```
        {
```

```
            PORTB = 0b00000000; // Clear all LEDs
```

```
            delay_ms (100);
```

```
            PORTB |= (1<< PINB2); // Turn on PB2
```

```
            delay_ms (100);
```

```
            PORTB |= (1<< PINB3); // Turn on PB3
```

```
PORTB |= (1<< PINB4); //
```

```
            delay_ms (100);
```

```
            PORTB |= (1<< PINB4); // Turn on PB4
```

```
            delay_ms (100);
```

```
        }
```

```
        // Right turn switch connected to PB1
```

```
        else if ( ( PINB & (1<< PINB1) ) == 0 ) // Active low on PB1
```

```
        {
```

```
            PORTB = 0b00000000; // Clear all LEDs
```

```
            delay_ms (100);
```

```
PORTB |= (1<< PINB5); // Turn on PB5
```

```
delay_ms(100);
```

```
PORTB |= (1<< PINB6); // Turn on PB6
```

```
delay_ms(100);
```

```
PORTB |= (1<< PINB7); // Turn on PB7
```

```
delay_ms(100);
```

```
}
```

```
else
```

```
{
```

```
PORTB = 0b 00000000; // Turn off all LEDs
```

```
}
```

```
}
```


Answer to the question no.2

(a) No, the ATmega 32 does not normally handle an interrupt within an interrupt. This is because the ATmega 32 automatically disables global interrupt when an interrupt service routine (ISR) is executed. The ATmega 32 disables the interrupt within an ISR is to ensure that the system does not encounter unexpected behaviour due to multiple ISRs trying to execute simultaneously. If multiple interrupts were allowed then a ~~more~~ higher priority interrupt may disrupt the current on going execution which further can cause data inconsistency.

During the execution of ISR, other interrupts can be allowed by manually enabling global interrupts within an ISR using the 'seiv' function. This 'seiv' function re-enables global interrupts, which further allows higher-priority or also lower-priority interrupts to occur.

(b)

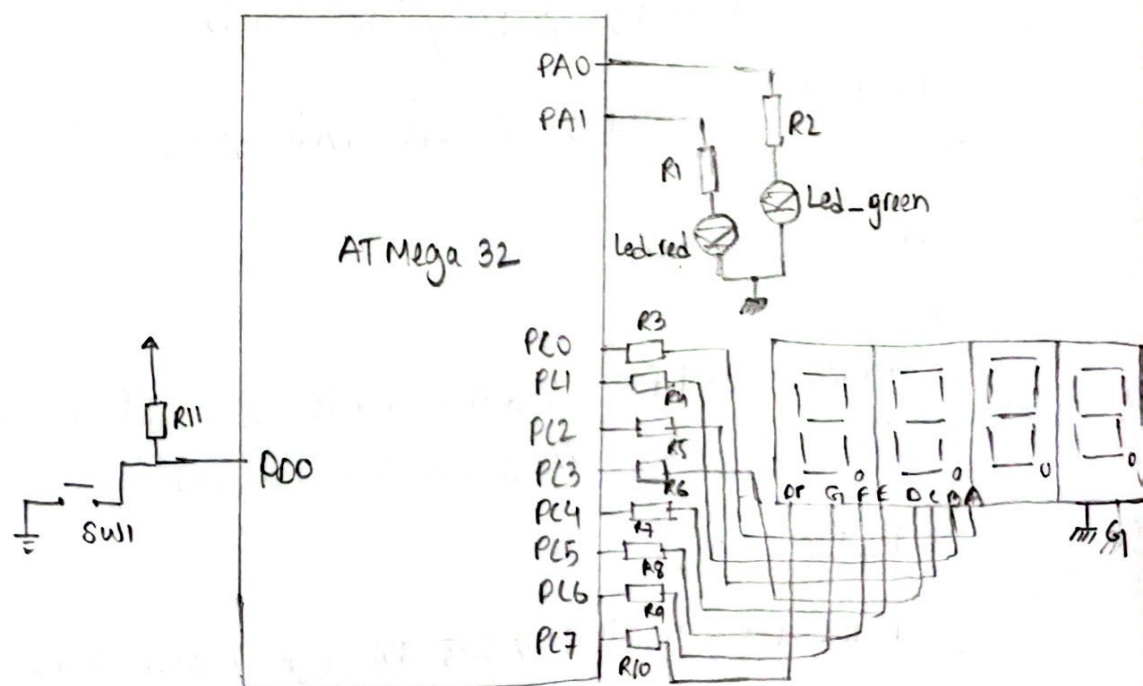


Fig: Circuit diagram of the given system

```
(c) #include <mega.h>
#include <delay.h>
```

```
int j;
```

```
// External interrupt INTO
```

```
: interrupt [EXT-INT0] void button(void) {
```

```
    unsigned char cathode[6] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6F};
```

```
    // 7 segment Hex codes for 0 to 5
```

```
    PORT A = 0x02; // sets PA1 (red) and clears PA0 (green)
```

```
    // Countdown on 7-segment display from 5 to 0
```

```
    for (j=5; j>0; j--)
```

```
    {
```

```
        PORT C = cathode[j] // Display countdown
```

```
        delay_ms(1000); // 1-second delay.
```

```
    }
```

```
    PORT C = 0; // clears 7-segment display
```

```
    delay_ms(10000); // delay 10 seconds
```

```
    PORT A = 0x01; // sets PA0 (green) and clears PA1 (red)
```

```
}
```

```
void main(void)
```

```
{
```

```
    DDRC = 0xFF; // define PORTC as output for 7-segment display
```

```
    DDRA = 0x03; // define PA0 and PA1 as outputs for LEDs
```

```
    PORT C = 0;
```

```
    PORT A = 0x01; // sets PA0 (green) and clears PA1 (red)
```

```
GICR |= (1 << INTO); // Enables external interrupt INTO
MCUCR |= (1 << ISC01); // Triggers interrupt on falling edge INTO

#asm ("sei") // Enables global interrupt

while (1)
{
    // Infinite loop waiting for interrupt
}
```



Course Code: EEE373
Course Title: Embedded System Design

Prepared by:

Afrida Islam
ID: 21221030
Section: 02

Assignment

4. Here, ID: 21221030

last digit $Y = 0$.

$$\therefore f = 1000 \text{ Hz}$$

$$T = \frac{1}{f} = \frac{1}{1000} = 10^{-3} \text{ secs}$$

(a) Given frequency of AT Mega 32 is, $f = 12 \text{ MHz}$

$$\therefore \frac{1200 \times 10^4}{\text{sec}} \times 1 \times 10^{-3} \text{ sec} \\ = 12000 \text{ cycle}$$

So to count 10^{-3} secs it needs 12000 cycles

Where as, Timer0 has, $2^8 - 1 = 255$ cycles.

$$\text{A prescaler of } 1024 = \frac{1024}{12 \times 10^6}$$

$$= 8.53 \times 10^{-5} \text{ secs}$$

In a square wave, the duty cycle gets half = $\frac{10^{-3}}{2}$

$$= 5 \times 10^{-4} \text{ secs}$$

$$\text{Hence, for half duty cycle, } = \frac{5 \times 10^{-4}}{8.53 \times 10^{-5}}$$

$$= 5.86 \approx 6 \text{ pulses}$$

Code:

```
#include <mega32.h>
```

```
interrupt [TIMO_COMP] void timer0_comp_isr (void)
```

```
{
```

```
    PORTA.0 = ~ PORTA.0 ; // toggles state.
```

```
    TCNT0 = 0; // initializes Timer 0 at 0.
```

```
}
```

```
void main (void)
```

```
{
```

```
    DDRA.0 = 1;
```

```
    TCCR0 = (0 << WGM00) | (0 << COM01) | (0 << COM00) |
```

```
            (0 << WGM01) | (1 << CS02) | (0 << CS01) |
```

```
            (1 << CS00); // Prescaler 1024 is set.
```

```
    TCNT0 = 0;
```

```
    OCR0 = 5; // Compare at 6th pulse
```

```
    TIMSK = 0b00000010;
```

```
#asm ("sei") // Enables global interrupt
```

```
    PORTC.0 = 0;
```

```
    while(1)
```

```
    {
```

```
        // runs infinite loop
```

```
    }
```

```
}
```


(b) without using any interrupt,

50% duty cycle gives = 0.5 ms [from 'a']

So, here we can toggle PORTA.0 at every 0.5 ms.

Code: #include <mega32.h>
#include <delay.h>

```
void main ()  
{  
    DDRA.0 = 1;  
    while (1)  
    {  
        PORTA.0 = ~ PORTA.0; // state changes at every  
        delay_ms (0.5) // 0.5 ms (high for 0.5 ms and low  
                        // for 0.5 ms)  
    }  
}
```


2. Given, frequency of PWM wave = 75 Hz

$$\therefore T = \frac{1}{f} = \frac{1}{75} = 0.0133 \text{ secs}$$
$$= 13.33 \text{ ms}$$

$$\text{At 60\% duty cycle} = 60\% \times 13.33$$
$$= 8 \times 10^{-3} = 8 \text{ ms.}$$

$$\text{A crystal with } 12\text{MHz} = \frac{12 \times 10^6}{\text{secs}} \times 8 \times 10^{-3} \text{ secs}$$
$$= 96000 \text{ cycles.}$$

Using Timer 2 with prescaler 1024 gives

$$f = \frac{12 \times 10^6}{1024} = 11718.75 \text{ Hz}$$

$$\therefore T_2 = \frac{1}{f} = 8.53 \times 10^{-5} \text{ secs}$$

60% of duty cycle
for that we need,

$$= \frac{8 \times 10^{-3}}{8.53 \times 10^{-5}}$$

$$= 93.75$$

$$\approx 94 \text{ pulses.}$$

$$\text{It remains high for } 8 \text{ ms and low for } = (13.33 - 8) \text{ ms}$$
$$= 5.33 \text{ ms}$$

$$\therefore \text{Now, we need, } = \frac{5.33 \times 10^{-3}}{8.53 \times 10^{-5}} = 62.52$$
$$\approx 63 \text{ pulses.}$$

Code: #include <mega32.h>
#include <delay.h>

interrupt [~~TM~~ TIM2_COMP] void timer2_comp_isr (void)
{
PORTA.0 = 1;
delay_ms (8) // PORT A stays high for 8 ms
PORTA.0 = 0; // stays low for rest 5.33 ms.
TCNT2 = 0;
}

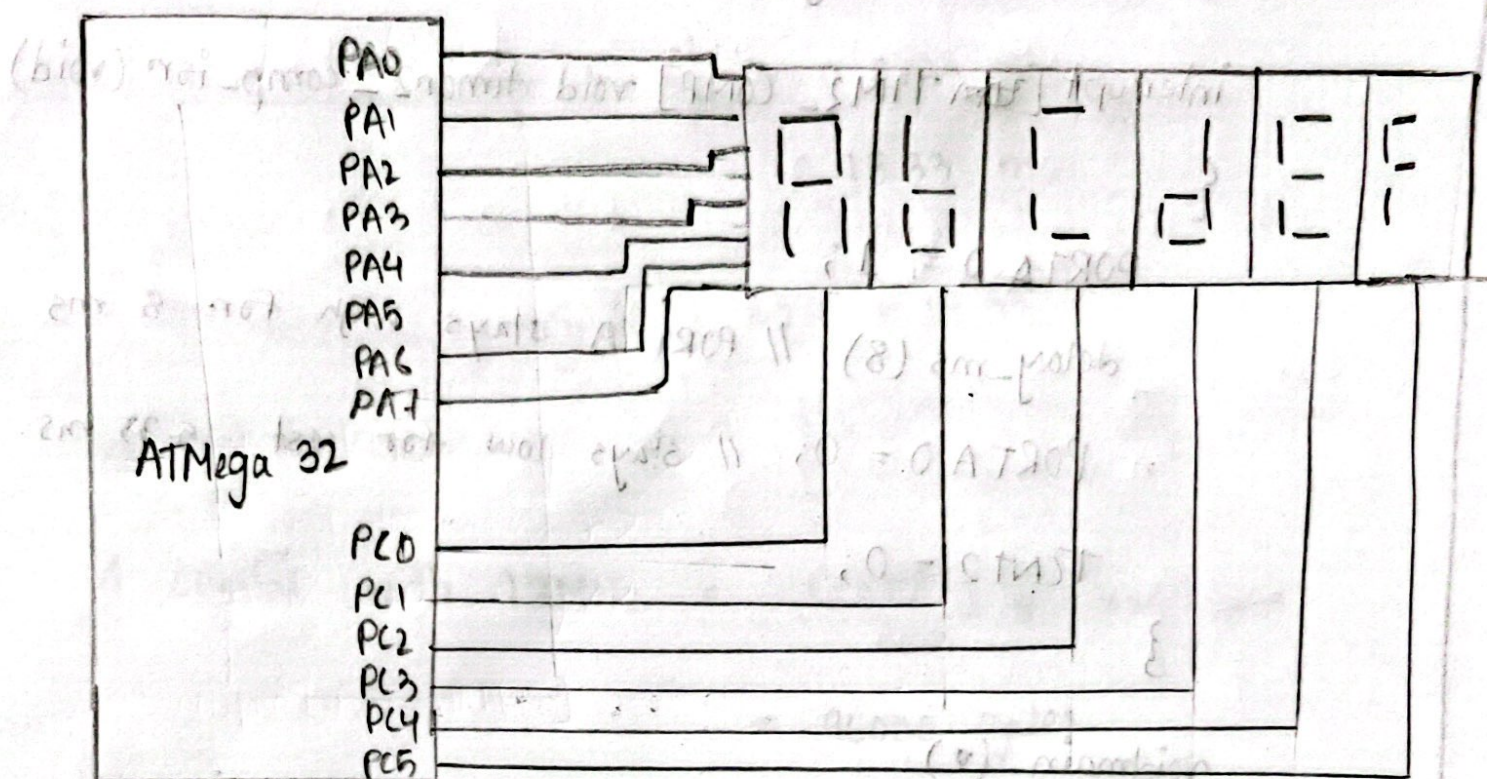
void main ()

{
DDRA.0 = 1;
ASSR = 1 << AS2; // External ^{crystal} is being used.
TCCR2 = (1 << CS22) | (1 << CS21) | (1 << CS20); // Prescaler of 1024 is set
TCNT2 = 0;
OCR2 = 0x3E; // Compares and raises flag at 63th pulse,
TIMSK = 1 << OCIE2; // hex code for 62 is given
#asm ("sei") // Enables global interrupt
PORTA.0 = 0; // initialized at 0.
while (1)
{
// runs infinite loop
}
}

3.

Circuit diagram:

<include> <include> #
<include> <include> #



To generate ABCDEF:

(7 segment) pin

a b c d e f g

hex code

0 1 1 1 0 1 1 1 → 0x77

0 1 1 1 0 0 0 0 → 0x7C

0 0 1 1 0 0 0 1 → 0x39

0 0 1 0 1 1 1 0 → 0x5D

0 1 1 1 1 0 0 1 → 0x79

0 1 1 1 0 0 0 1 → 0x71


```

Code: #include <mega32.h>
#include <stdio.h>
#include <delay.h>

```

```

void main ( )

```

```

{

```

```

    DDRA = 0x FF; // Data port A

```

```

    PORT DDRC = 0x FF; // Control port C

```

```

    while (1)
    {

```

```

        PORT C = 0b11111110; // LED output in seven segment

```

```

        PORTA = 0x 77; // Sets A

```

```

        delay-ms (10);

```

```

        PORTC = 0b11111101;

```

```

        PORTA = 0x 7C; // Sets b

```

```

        delay-ms (10)

```

```

        PORTC = 0b11111011;

```

```

        PORTA = 0x 39; // Sets C

```

```

        delay-ms (10)

```

```

        PORTC = 0b11110111;

```

```

        PORTA = 0x 5D; // Sets d

```

```

        delay-ms (10)

```

```

        PORTC = 0b11101111;

```

```

        PORTA = 0x 79; // Sets E

```

```

        delay-ms (10)
    }
}

```


PORTC = 0b11011111; <#include <mega32.h> #

PORTA = 0x71; // sets <f.oibte> #

delay_ms(10) <#include <delay.h> #

}

}

4. #include <mega32.h>

#include <stdio.h>

#include <delay.h>

interrupt [EXT_INT0] void ext_int0_isr(void)

{

PORTA = 0xF0;

delay_ms(20000) // interrupts for 20 seconds when

}

switch is pressed.

void main()

{

DDRA = 0xFF;

GICR = (0<<INT1) | (1<<INT0) | (0<<INT2) // Enables INT0.

MCUCR = (0<<ISC11) | (0<<ISC10) | (1<<ISC01) | (0<<ISC00)

// generates interrupt when INT0 changes

#asm("sei") // Enables global interrupt.

while(1) // infinite loop generation

{

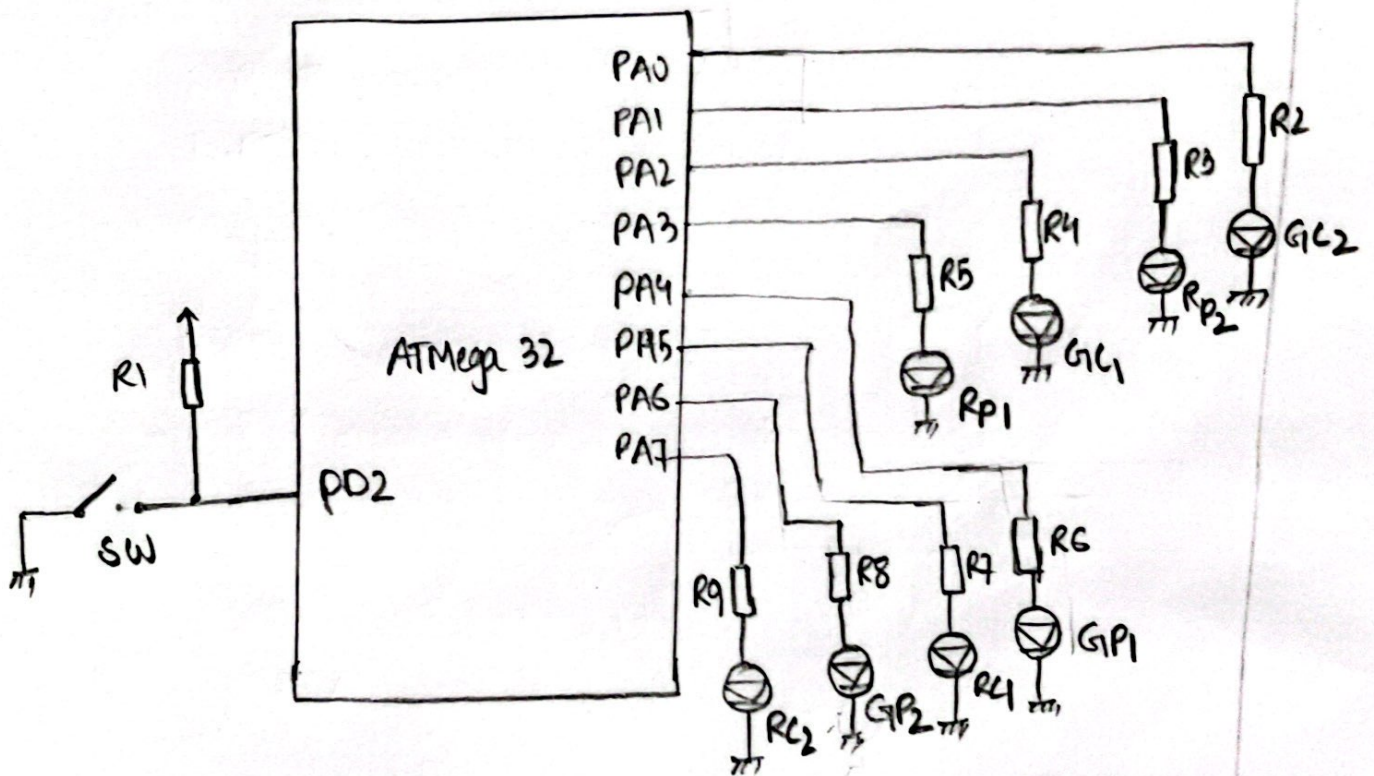
PORTA = 0x0F; // normal LED operation

delay-ms (45000); // 45 seconds ON for GL1, GL2, RP1, RP2

PORT A = 0x F0;

delay-ms (15000) // 15 seconds ON for RL1, RL2, GP1, GP2

Circuit for this system:



Here,

GL1, GL2, GP1, GP2 → set as LED-GREEN

RP1, RP2, RL1, RL2 → set as LED-RED.

EEE373L

Embedded System Design Laboratory

Lab Report



Inspiring Excellence

Section:02

Group No:02

Experiment no:02

Name of the experiment: **Fundamentals of Embedded C**

Prepared by:

Name: Afrida Islam

ID: 21221030 **Signature**

All Group members:

Sl.	Name	ID	Signature
1.	Yasir Anaf Siddique	21221073	Anaf
2.	Md Talha Bin Imran	21221018	Talha
3.	Md. Hamin-Al-Araf	21221019	Araf
4.	Nafim Karim Khan	21221039	Nafim

Date of Submission: 30/06/2024

Objective:

The objective of this experiment is to illustrate the foundations of programming in the Embedded C language.

Codes:

Part A:

```
#include <mega32.h> // library for ATmega32 microcontroller
#include <alcd.h>    // library for LCD functions

unsigned char globey; // A global unsigned character variable 'globey' declared

void function_z (void) // A function_z is defined
{
    unsigned int tween; //A local unsigned integer variable 'tween' declared
    tween = 12; // Assign the value 12 to 'tween'
    globey = 47; // Assign the value 47 to the global variable 'globey'
    main_loc = 12; // 'main_loc' is assigned as 12
}

void main(void) // The main function is defined
{
    unsigned char main_loc; //A local unsigned character variable 'main_loc'
    declared
    lcd_init(16); // Initializing the LCD with 16 columns
    globey = 34; // The value 34 is assigned to the global variable 'globey'
    tween = 12; // 12 is assigned to 'tween'
    while (1) // Generates infinite loop
    {
    }
}
```

Discussion:

There are two errors generated because the variable `main\_loc` is used in `function\_z`, but it is not declared within the scope of the function. This results in an error because the function has no access to the `main\_loc` declared in the `main` function. Again, the variable `tween` is used in the `main` function without being declared within its scope. This results in an error because `tween` is not accessible outside of `function\_z`, where it is declared locally.

Part B:

Code:

```
#include <mega32.h>    // library for ATmega32 microcontroller
#include <alcd.h>       // library for LCD functions
#include <delay.h>      // library for delay functions
#include <stdlib.h>     // standard library for functions like itoa
#include <string.h>     // string library for functions like strcpy and strcat

char k; // A global character variable 'k' is declared

void display(char var, char disp1[16]) // Function to display a variable on the
LCD
{
    char disp2[16];          // Declares a local character array 'disp2' of size
16
    itoa(var,disp2);         // Converts the integer 'var' to a string and store
it in 'disp2'
    strcat(disp1,disp2);     // Concatenates 'disp1' and 'disp2'
    lcd_gotoxy(0,1);         // Sets the cursor to the first position of the
second row on the LCD
    lcd_puts(disp1);         // Displays the string 'disp1' on the LCD
}

void main(void) // Declares main function
```



```

{
    char i, disp1[16]; // Declares local variables 'i' and 'disp1' (array of
size 16)

    lcd_init(16);      // Initializes the LCD with 16 columns

    while (1) // Generates infinite loop
    {
        i = 1; // Initializes 'i' with 1

        k = 4 * i++; // Assigns 'k' the value of 4 multiplied by 'i', then
increment 'i'

        lcd_clear(); // Clears the LCD

        lcd_gotoxy(0,0); // Sets the cursor to the first position of the first
row on the LCD

        lcd_putsf("Post-incr "); // Displays the string "Post-incr" on the LCD
strcpy(disp1, "k = "); // Copies the string "k = " into 'disp1'

        display(k, disp1); // Calls the display function to show 'k' on the LCD
delay_ms(1000); // Delays for 1000 milliseconds

        i = 1; // Initializes 'i' with 1

        k = 4 * ++i; // Increments 'i' first, then assigns 'k' the value of 4
multiplied by 'i'

        lcd_gotoxy(0,0); // Sets the cursor to the first position of the first
row on the LCD

        lcd_putsf("Pre-incr"); // Displays the string "Pre-incr" on the LCD
strcpy(disp1, "k = "); // Copies the string "k = " into 'disp1'

        display(k, disp1); // Calls the display function to show 'k' on the LCD
delay_ms(1000); // Delays for 1000 milliseconds

        // i += 2; //increments 'i' by 2

        k /= i; // Divides 'k' by 'i' and assign the result to 'k' (k = 4)

        k |= 5; // Performs bitwise OR on 'k' with 5 and assign the result to
'k' (k = 5)

```

```
k &= 3; // Performs bitwise AND on 'k' with 3 and assign the result to
'k' (k = 1)

k ^= 7; // Performs bitwise XOR on 'k' with 7 and assign the result to
'k' (k = 6)

lcd_clear(); // Clears the LCD

lcd_gotoxy(0,0); // Sets the cursor to the first position of the first
row on the LCD

lcd_putsf("Compound "); // Displays the string "Compound" on the LCD
strcpy(displ, "k = "); // Copies the string "k = " into 'displ'
display(k, displ); // Calls the display function to show 'k' on the LCD
delay_ms(1000); // Delays for 1000 milliseconds

}

}
```

Output:

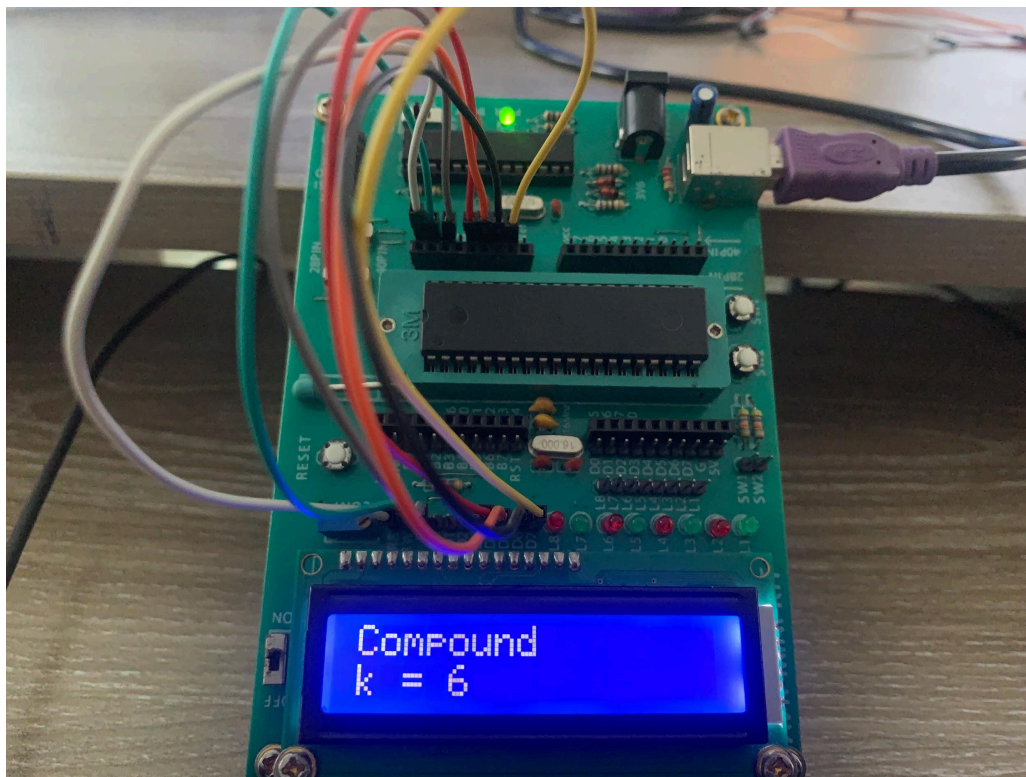


Figure-1: Output using Assignment statement

Part C:

Code:

```
#include <mega32.h>    // library for ATmega32 microcontroller
#include <alcd.h>       // library for LCD functions
#include <stdlib.h>     // standard library for functions like itoa
#include <string.h>     // string library for functions like strcpy and strcat
#include <delay.h>      // library for delay functions

void display(char var, char disp1[16]) // Function to display a variable on the
LCD
{
    char disp2[16];        // Declares a local array 'disp2' of size 16
    itoa(var, disp2);      // Converts the integer 'var' to a string and store
it in 'disp2'
    strcat(disp1, disp2); // Concatenates 'disp1' and 'disp2'
    lcd_gotoxy(0, 1);     // Sets the cursor to the first position of the
second row on the LCD
    lcd_puts(disp1);       // Displays the string 'disp1' on the LCD
}

void main(void) // Declares Main function
{
    char i, k, disp1[16]; // Declares local variables 'i', 'k', and 'disp1'
(array of size 16)
    lcd_init(16);         // Initializes the LCD with 16 columns

    while (1) // Generates infinite loop
    {
        i = 0; // Initializes 'i' with 0
        k = 0; // Initializes 'k' with 0
```

```

// While loop upto 20

while (i < 20) // Loop until 'i' is less than 20
{
    k += 3; // Increments 'k' by 3
    i++;    // Increments 'i' by 1
}

    lcd_gotoxy(0, 0); // Sets the cursor to the first position of the
first row on the LCD

    lcd_puts("while loop..."); // Displays the string "while loop..." on
the LCD

    strcpy(displ, "k = "); // Copies the string "k = " into 'displ'
    display(k, displ); // Calls the display function to show 'k' on the LCD
    delay_ms(1000); // Delays for 1000 milliseconds

    i = 0; // Reinitializes 'i' with 0
    k = 0; // Reinitializes 'k' with 0

// Do-while loop starts
do // Execute the loop body at least once
{
    k += 3; // Increments 'k' by 3
    i++;    // Increments 'i' by 1
}

while (i < 20); // Continues looping until 'i' is less than 20

    lcd_gotoxy(0, 0); // Sets the cursor to the first position of the first
row on the LCD

```

```
LCD

lcd_puts("do while ..."); // Displays the string "do while ..." on the

strcpy(displ, "k = "); // Copies the string "k = " into 'displ'

display(k, displ); // Calls the display function to show 'k' on the LCD

delay_ms(1000); // Delays for 1000 milliseconds

}

}
```

Output:



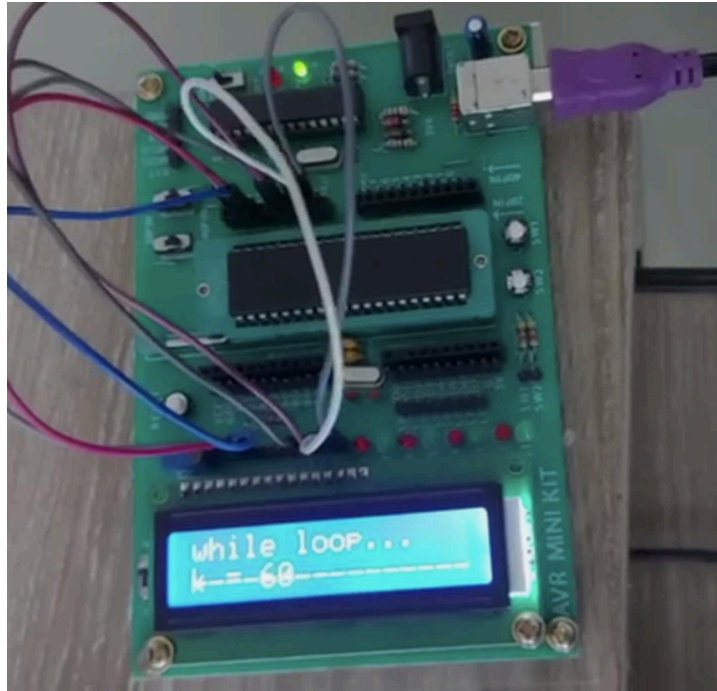


Figure-2,3: Output using While and Do While loop

Part D:

Code:

```
#include <mega32.h>    // library for ATmega32 microcontroller
#include <alcd.h>       // library for LCD functions
#include <stdlib.h>     // standard library for functions like itoa
#include <string.h>     // string library for functions like strcpy and strcat
#include <delay.h>     // library for delay functions

// Function to display a variable on the LCD
void display(char var, char disp1[16])
{
    char disp2[16];      // Declares a local array 'disp2' of size 16
    itoa(var, disp2);    // Converts the integer 'var' to a string and store
                        // it in 'disp2'
    strcat(disp1, disp2); // Concatenates 'disp1' and 'disp2'
```

```

    lcd_gotoxy(0, 1);      // Sets the cursor to the first position of the
second row on the LCD

    lcd_puts(displ);      // Displays the string 'displ' on the LCD
}

void main(void) // Declares Main function
{
    char i, k, displ[16]; // Declares local variables 'i', 'k', and 'displ'
(array of size 16)

    lcd_init(16);         // Initializes the LCD with 16 columns

    while (1) // Generates infinite loop
    {
        k = 0; // Initializes 'k' with 0

        // For loop with incrementing 'i'
        for (i = 0; i < 20; i++) // Loop from 0 to 19
        {
            k += 3; // Increments 'k' by 3 in each iteration
        }

        lcd_clear();      // Clears the LCD

        lcd_gotoxy(0, 0); // Sets the cursor to the first position of
the first row on the LCD

        lcd_puts("for loop (++)..."); // Displays the string "for loop (++)..."
on the LCD

        strcpy(displ, "k = "); // Copies the string "k = " into 'displ'

        display(k, displ); // Calls the display function to show 'k' on
the LCD

        delay_ms(1000);    // Delays for 1000 milliseconds
    }
}

```

```

k = 60; // Initializes 'k' with 60

// For loop with decrementing 'i'
for (i = 20; i > 0; i--) // Loop from 20 to 1
{
    k -= 3; // Decrements 'k' by 3 in each iteration
}

lcd_clear(); // Clears the LCD

lcd_gotoxy(0, 0); // Sets the cursor to the first position of
the first row on the LCD

lcd_puts("for loop (--)..."); // Displays the string "for loop (--)..."
on the LCD

strcpy(displ, "k = "); // Copies the string "k = " into 'displ'

display(k, displ); // Calls the display function to show 'k' on
the LCD

delay_ms(1000); // Delays for 1000 milliseconds
}
}

```


Output:

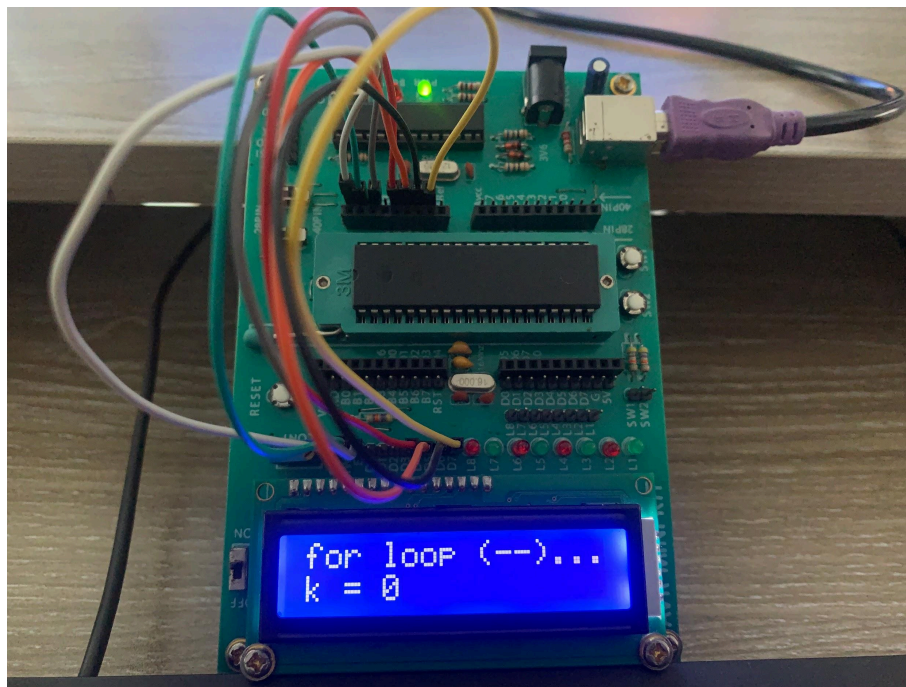


Figure-4,5: Output using For loop

Part E:

Code:

```
#include <mega32.h>    // library for ATmega32 microcontroller
#include <alcd.h>       // library for LCD functions
```

```

#include <stdlib.h>    // standard library for functions like itoa
#include <string.h>    // string library for functions like strcpy and strcat
#include <delay.h>     // library for delay functions

// Function to display a variable on the LCD
void display(char var, char disp1[16])
{
    char disp2[16];    // Declares a local array 'disp2' of size 16
    itoa(var, disp2);  // Converts the integer 'var' to a string and store
it in 'disp2'

    strcat(disp1, disp2); // Concatenates 'disp1' and 'disp2'

    lcd_gotoxy(0, 1);   // Sets the cursor to the first position of the
second row on the LCD

    lcd_puts(disp1);    // Displays the string 'disp1' on the LCD
}

void main(void) // Declares main function
{
    char i, k, disp1[16]; // Declares local variables 'i', 'k', and 'disp1'
(array of size 16)

    lcd_init(16);        // Initializes the LCD with 16 columns

    while (1) //Generates infinite loop
    {
        // if-else if-else statement

        i = 15; // Initializes 'i' with 15

        k = 7;  // Initializes 'k' with 7

        lcd_clear(); // Clears the LCD

        lcd_gotoxy(0, 0); // Sets the cursor to the first position of the
first row on the LCD

```

```

    if (i < 10) // If 'i' is less than 10
    {
        lcd_puts("if i<10 ....."); // Displays the string "if i<10
        ..... " on the LCD

        k += 10; // Increments 'k' by 10
    }

    else if (i >= 10 && i < 20) // If 'i' is between 10 and 19 inclusive
    {
        lcd_puts("if i>=10&&i<20.."); // Displays the string "if
        i>=10&&i<20.." on the LCD

        k += 15; // Increments 'k' by 15
    }

    else // If 'i' is 20 or more
    {
        lcd_puts("if i>=20 ....."); // Displays the string "if i>=20
        ..... " on the LCD

        k += 20; // Increments 'k' by 20
    }

    strcpy(displ, "k = "); // Copies the string "k = " into 'displ'
    display(k, displ);      // Calls the display function to show 'k' on the
LCD

    delay_ms(1000);        // Delays for 1000 milliseconds

    // Switch statement

    i = 1; // Initializes 'i' with 1
    k = 7; // Initializes 'k' with 7

    lcd_clear();           // Clears the LCD

```



```
    lcd_gotoxy(0, 0);        // Sets the cursor to the first position of the
first row on the LCD
```

```
    switch (i) // Switches statement based on the value of 'i'
    {
        case 1: // If 'i' is 1

            lcd_puts("switch i=1 ....."); // Displays the string "switch
i=1 ..... " on the LCD

            k += 10; // Increments 'k' by 10

            break;    // Exits the switch statement

        case 2: // If 'i' is 2

            lcd_puts("switch i=2....."); // Displays the string "switch
i=2....." on the LCD

            k += 15; // Increments 'k' by 15

            break;    // Exits the switch statement

        case 3: // If 'i' is 3

            lcd_puts("switch i=3....."); // Displays the string "switch
i=3....." on the LCD

            k += 20; // Increments 'k' by 20

            break;    // Exits the switch statement

        default: // If 'i' is any other value

            lcd_puts("switch default...."); // Displays the string "switch
default...." on the LCD

            k += 0; // Do not change the value of 'k'

            break;    // Exits the switch statement
    }

    strcpy(displ, "k = "); // Copies the string "k = " into 'displ'

    display(k, displ);      // Calls the display function to show 'k' on the
LCD

    delay_ms(1000);        // Delays for 1000 milliseconds
```

}

}

Output:

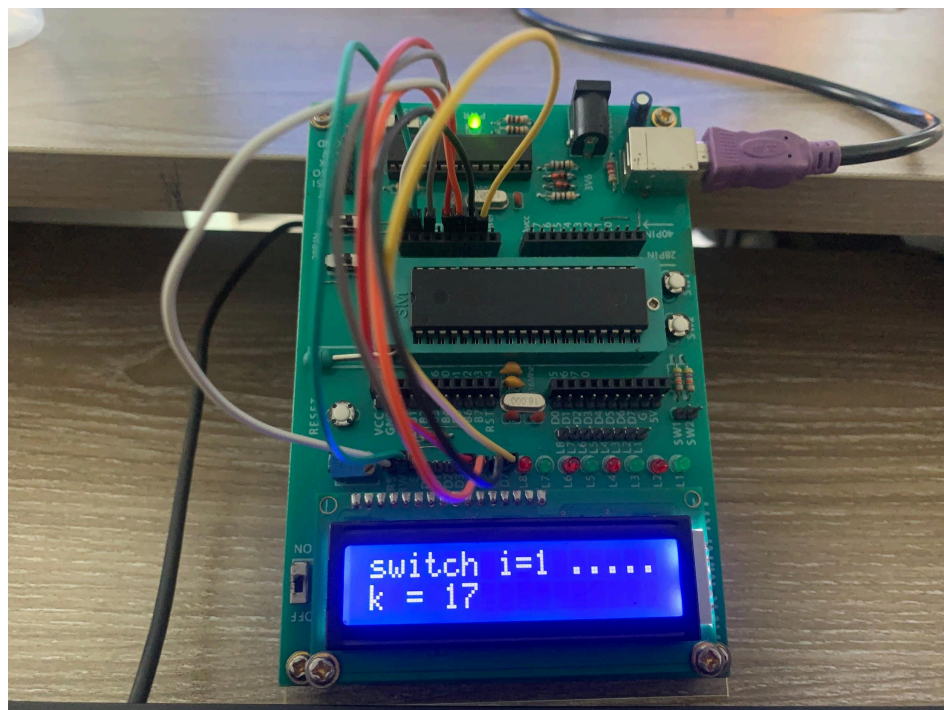


Figure-6,7: Output using if else and switch statement

EEE373L

Embedded System Design Laboratory

Lab Report



Inspiring Excellence

Section:02

Group No:02

Experiment no:03

Name of the experiment:Input / Output of Digital Data

Prepared by:

Name: Nafim Karim Khan

ID: 21221039

Signature

All Group members:

Sl.	Name	ID	Signature
1.	Yasir Anaf Siddique	21221073	
2.	Md Talha Bin Imran	21221018	
3.	Afrida Islam	21221030	
4	Md. Hamin-Al-Araf	21221019	

Date of Submission: 22/06/2024

Objective

The objective of this experiment is to demonstrate inputting / outputting of digital data from / to the microcontroller ATmega32.

Theoretical Background

Risc (Reduced Instruction Set Computer) architecture is the foundation of the AVR microcontroller. An example of a low power CMOS controller is the ATmega32 microcontroller. AVR microcontrollers with RISC architecture are capable of processing one million instructions per second when given a crystal oscillator's cycle frequency of 1 MHz. The ATmega32 chip has 40 pins. For the four ports—PORTA, PORTB, PORTC, and PORTD—a total of 32 pins are reserved. It is possible to use these ports for both data input and output. The remaining pins are labeled as AVCC, AREF, AGND, XTAL1, XTAL2, RESET, GND, and VCC. Any of these ports must be configured appropriately in order to be used as an input or output port.

Part A: Sending Data to a Port

Circuit Diagram

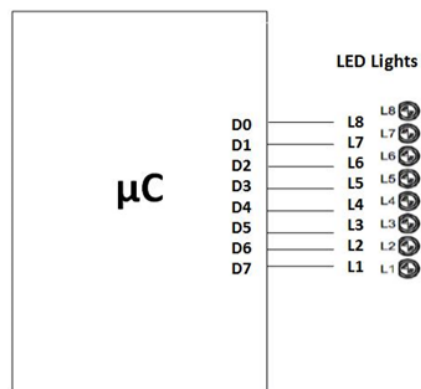


Fig.1: Circuit configuration of part A

Code

```
#include <mega32.h>
#include <delay.h>
void main(void)
{
    DDRD=0xFF;
    PORTD=0xFF;
    while (1)
```

```
{  
  delay_ms(1000);  
  PORTD=~PORTD;  
}
```

Output

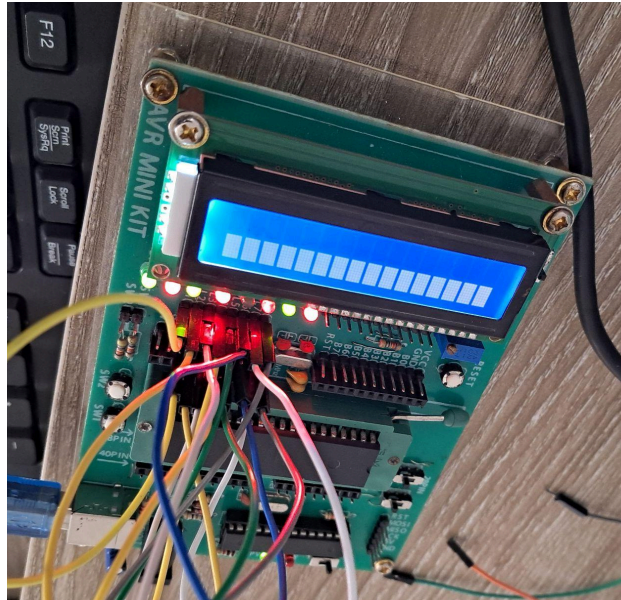


Fig 2: All LEDs ON

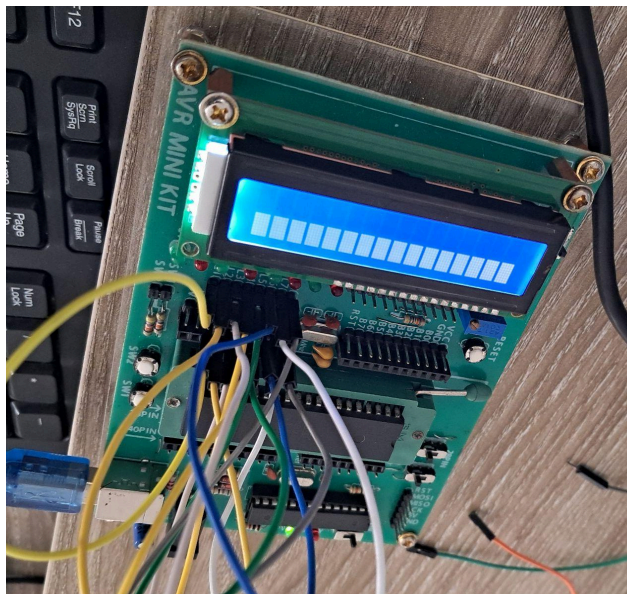


Fig 3: All LEDs OFF

Discussion

In this task, we observe how a particular set of ports can be configured to be output pins and how we can send data to these ports. Here the LEDs are connected to PORT D and are set as outputs by denoting all the PORTD to High (1). The output task shows 8 LEDs blinking with an interval of 1 sec.

Lab Report Task

Write and explain the modified program so that it will illuminate the alternate LEDs every second.

Code

```
#include <mega32.h>
#include <delay.h>

void main(void)
{
    DDRD = 0xFF;      // Set PORTD as output
    PORTD = 0x00;      // Initialize all LEDs to OFF

    while (1)
    {
        PORTD = 0xAA; // Turn on alternate LEDs (10101010)
        delay_ms(1000);

        PORTD = 0x55; // Turn on the other alternate LEDs (01010101)
        delay_ms(1000);
    }
}
```

Discussion

The modified program is designed to illuminate alternate LEDs every second on PORTD. First, DDRD = 0xFF sets all the pins of PORTD as output, meaning the program can control the LEDs connected to these pins. Initially, all LEDs are turned off by setting PORT D = 0x00. Inside the while loop, the program alternates between two patterns: PORTD = 0xAA (binary 10101010), which turns on the odd-numbered LEDs, and PORT D = 0x55 (binary 01010101), which turns on the even-numbered LEDs. A delay_ms(1000) function is used to introduce a one-second delay between switching the LED states, making the LEDs alternate every second.

Part-B: Multiple 7 segment display

Circuit Diagram

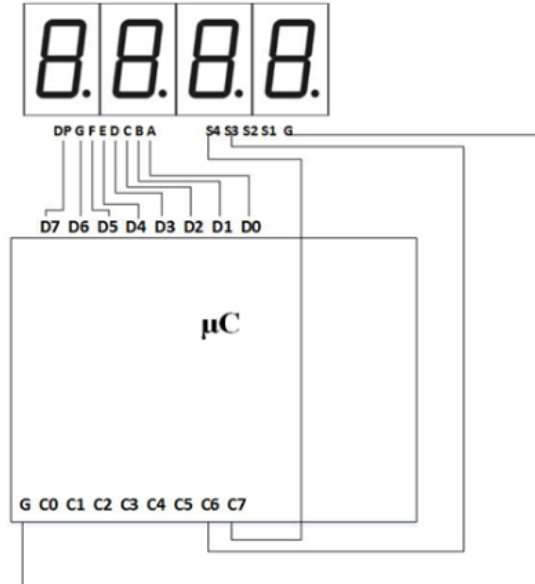


Fig 4: Circuit configuration of part B

Code

```
#include <mega32.h>
#include <delay.h>
#define data_port PORTD
#define control_port PORTC
char digit_cathode[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
char first,last,i=0,j;
void main(void)
{
    DDRD=0xFF;
    DDRC=0xC0;
    data_port=0;
    control_port=0;
while (1)
{
    last=i%10; //last digit
    first=i/10; //first digit
    for(j=0;j<20;j++)
    {
        control_port=0x40; //0b01000000; S3(C6) is for the 2nd last digit
        data_port=digit_cathode[first];
        delay_ms(20);
        control_port=0x80; //S4(C7) is for the last digit
        data_port=digit_cathode[last];
        delay_ms(20);
    }
    i++;
    if (i>99) i=0;
}
```


}
}

Output

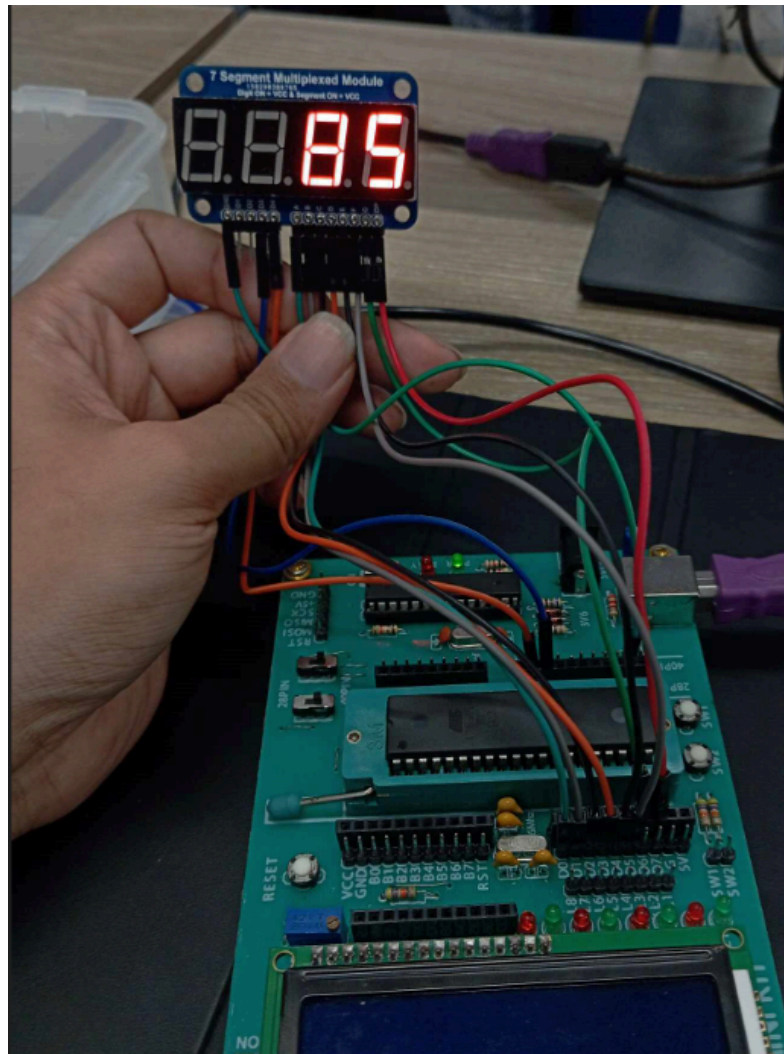


Fig 5.1: 7-Seg Display counting down from 0-99

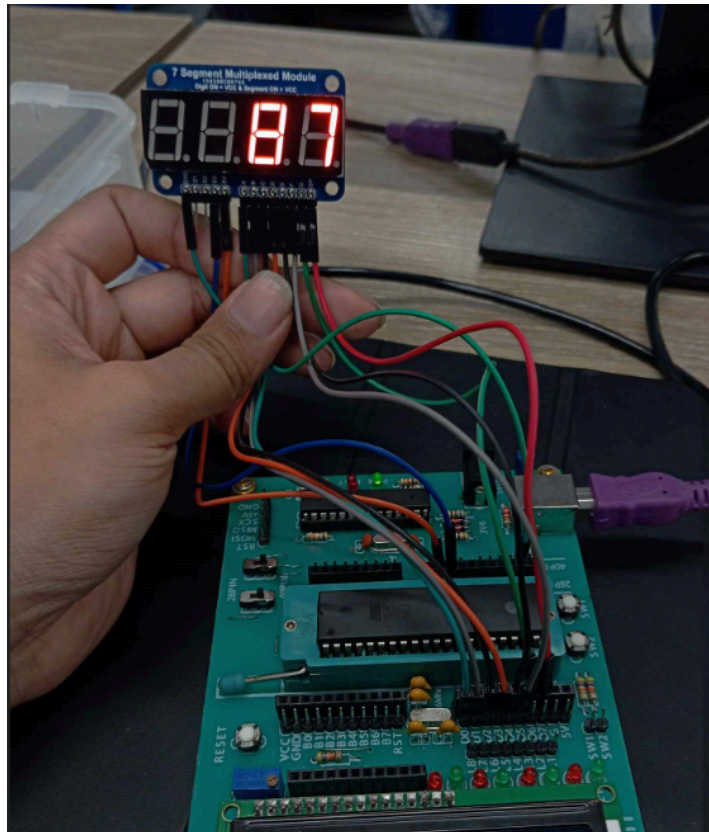


Fig 5.2: 7-Seg Display counting down from 0-99 (Continued)

Discussion

This code uses an ATmega32 microcontroller to drive a two-digit, seven-segment display. The segment patterns used to show digits 0 through 9 are stored in the array `{digit_cathode}`. The segment data is managed by `data_port` (PORTD), while `control_port` (PORTC) alternates between the two digits. Through multiplexing the display and updating each digit every 20 ms, the software shows the first and last digits of a counter (up to 99). With every loop, the `{i}` variable becomes bigger and bigger until it reaches 99, at which point it resets to 0. It then counts up to 99 and keeps the numbers on the screen.

Report Task

Write and explain a modified program so that it will display 000-999.

Code

```
#include <mega32.h>
#include <delay.h>
#define data_port PORTD
#define control_port PORTC
char digit_cathode[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F}; // Segment
codes for digits 0-9
char first, second, last, i = 0, j;
```

```

void main(void)
{
    DDRD = 0xFF;      // Set PORTD as output
    DDRC = 0xE0;      // Set PORTC (C5, C6, C7) as output

    data_port = 0;
    control_port = 0;

    while (1)
    {
        last = i % 10;      // Get the last digit (ones)
        second = (i / 10) % 10; // Get the second digit (tens)
        first = i / 100;    // Get the first digit (hundreds)

        for (j = 0; j < 20; j++)
        {
            // Control for the first digit (hundreds place)
            control_port = 0x20; // 0b00100000; S2 (C5) controls hundreds
            data_port = digit_cathode[first];
            delay_ms(10);

            // Control for the second digit (tens place)
            control_port = 0x40; // 0b01000000; S3 (C6) controls tens
            data_port = digit_cathode[second];
            delay_ms(10);

            // Control for the last digit (ones place)
            control_port = 0x80; // 0b10000000; S4 (C7) controls ones
            data_port = digit_cathode[last];
            delay_ms(10);
        }

        i++; // Increment the counter
        if (i > 999) i = 0; // Reset after reaching 999
    }
}

```

Discussion

This modified program controls a three-digit 7-segment display, allowing it to count from 000 to 999. The digits are extracted from the counter `i`, with `first` representing the hundreds, `second` the tens, and `last` the ones. The display is multiplexed by activating each digit in quick succession using `control\_port` (C5, C6, C7). Each digit is displayed for 10 ms, making all digits appear lit simultaneously. The counter increments from 0 to 999, resetting after reaching 999, displaying numbers continuously.

Part C: Input of digital data

Circuit Diagram

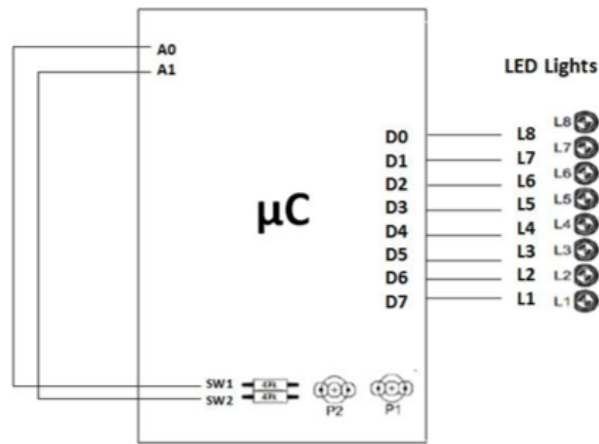


Fig.6: Circuit configuration of part C

Code

```
#include <mega32.h>
#include <delay.h>
void main(void)
{
    char move=0;
    DDRD=0xFF;
    PORTD=0x80;
    DDRA=0x00;
    PORTA=0x03; //to turn on the pull up resistor
    while (1)
    {
        if (PINA.0==0) move=1;
        if (PINA.1==0) move=2;

        if (move==1) //if (PINA.0==0)
        {
            PORTD=PORTD>>1;
            if (PORTD==0)
                PORTD=0x80;
        }
        if (move==2)
        {
            PORTD=PORTD<<1;
            if (PORTD==0)
                PORTD=0x01;
        }
        delay_ms(200);
    }
}
```


Output

- When Switch 1 is pressed, LEDs shift right

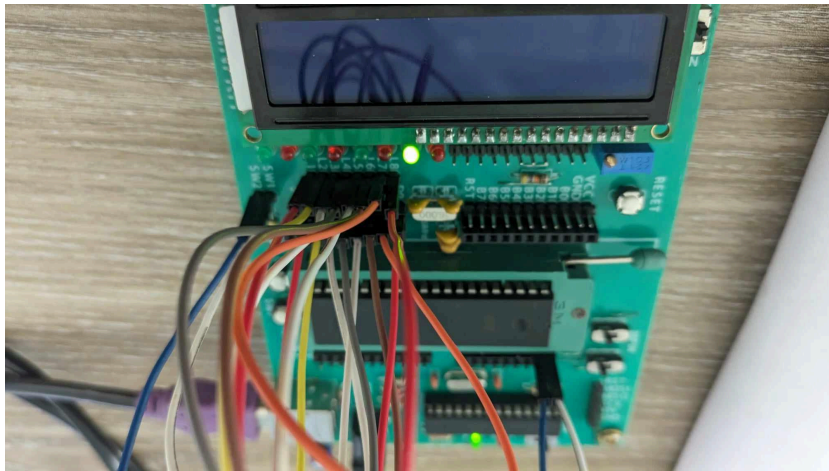


Fig 6.1: LED Shifting right when SW1 is Pressed

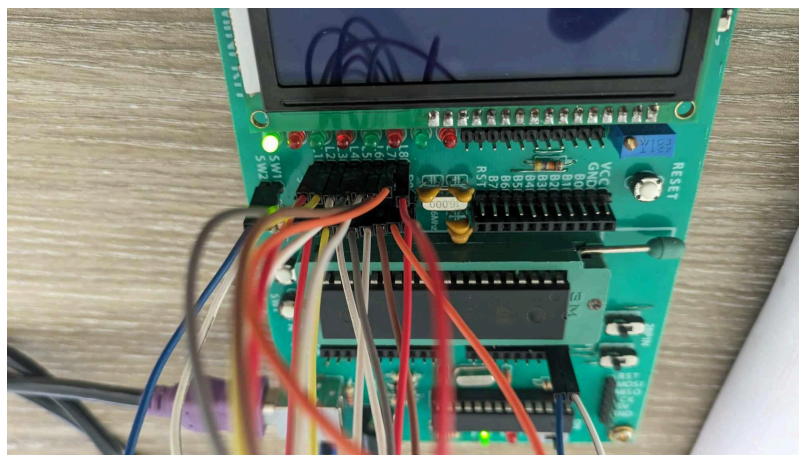


Fig 6.2: LED Shifting right when SW1 is Pressed (Continued)

- When Switch 2 is pressed, LEDs shift left

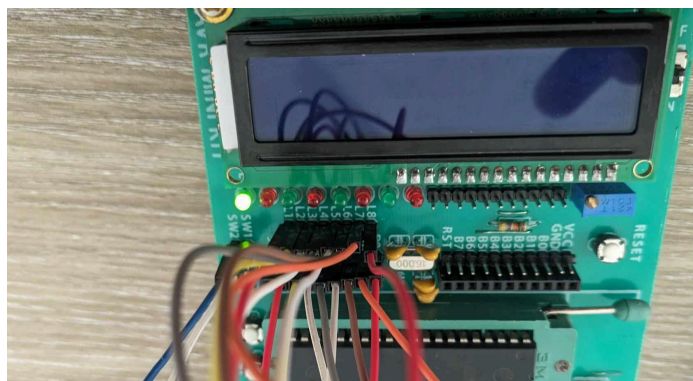


Fig 6.3: LED shifting leftwards when SW2 Pressed

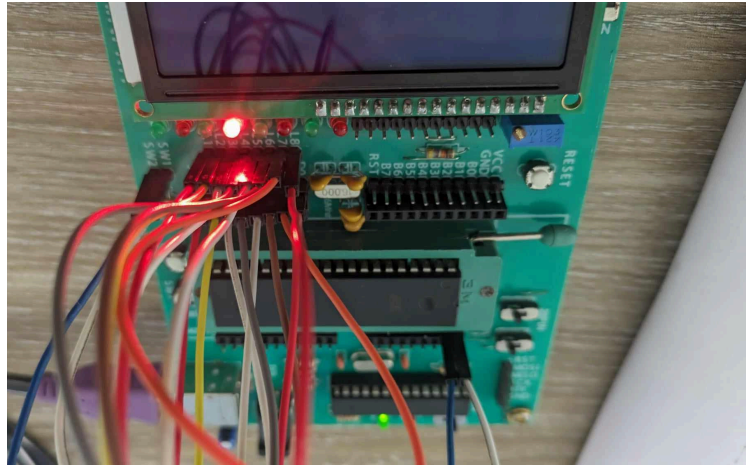


Fig 6.4: LED shifting leftwards when SW2 pressed (Continued)

Discussion

This program controls the shifting of an LED on 'PORTD' based on the input from two push buttons connected to 'PINA.0' and 'PINA.1'. When button 'PINA.0' is pressed, the LED shifts right, and when button 'PINA.1' is pressed, the LED shifts left. The pull-up resistors on 'PINA' are enabled, ensuring stable button readings. The 'PORTD' value shifts either left or right by one bit, wrapping around once it reaches the end. The LED shift operation occurs every 200 milliseconds, making the movement visible.

Report task

Write and explain a modified program so that it will reverse the order of LED shift operation.

Code

```
#include <mega32.h>
#include <delay.h>

void main(void)
{
    char move = 0;
    DDRD = 0xFF;           // Set PORTD as output
    PORTD = 0x01;          // Start with the first LED on (right-most)
    DDRA = 0x00;           // Set PORTA as input
    PORTA = 0x03;          // Enable pull-up resistors on PINA.0 and PINA.1

    while (1)
    {
        if (PINA.0 == 0) move = 1; // Button PINA.0 pressed
        if (PINA.1 == 0) move = 2; // Button PINA.1 pressed

        if (move == 1) // Move left when PINA.0 is pressed
        {
```

```

        PORTD = PORTD << 1;      // Shift left
        if (PORTD == 0)          // Reset if no LEDs are on
            PORTD = 0x01;
    }
    if (move == 2) // Move right when PINA.1 is pressed
    {
        PORTD = PORTD >> 1;      // Shift right
        if (PORTD == 0)          // Reset if no LEDs are on
            PORTD = 0x80;
    }
    delay_ms(200); // Delay between shifts
}
}

```

Discussion

This modified program reverses the direction of the LED shift operations. Initially, PORTD is set to 0x01, turning on the right-most LED. When button PINA.0 is pressed, the LED shifts left (PORTD << 1), and when button PINA.1 is pressed, the LED shifts right (PORTD >> 1). If the LED reaches either end (no LEDs are on), it wraps around to the opposite end: PORTD = 0x01 (right-most) when shifting left, and PORTD = 0x80 (left-most) when shifting right. The 200 ms delay controls the speed of the shifts.

Bonus Class Task

- Display HIGH on 7 Segment display

The Code

```

#include <mega32.h>
#include <delay.h>

char digit_cathode[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
void main(void)
{
    DDRC=0xFF; //used for output data
    DDRD=0x0F; //last 4 LSB used for switching displays, also acting as outputs

    while (1){
        PORTC=0x76; //Hex code for displaying 'H' on CC 7 segment display
        PORTD=0b00000001;
        delay_ms(10);
        PORTC=0x7D; //G
        PORTD=0b00000010;
        delay_ms(10);
        PORTC=0x06; //I
        PORTD=0b00000100;
        delay_ms(10);

        PORTC=0x76; //H
        PORTD=0b00001000;
    }
}

```

```
delay_ms(10);  
}  
}
```

Output

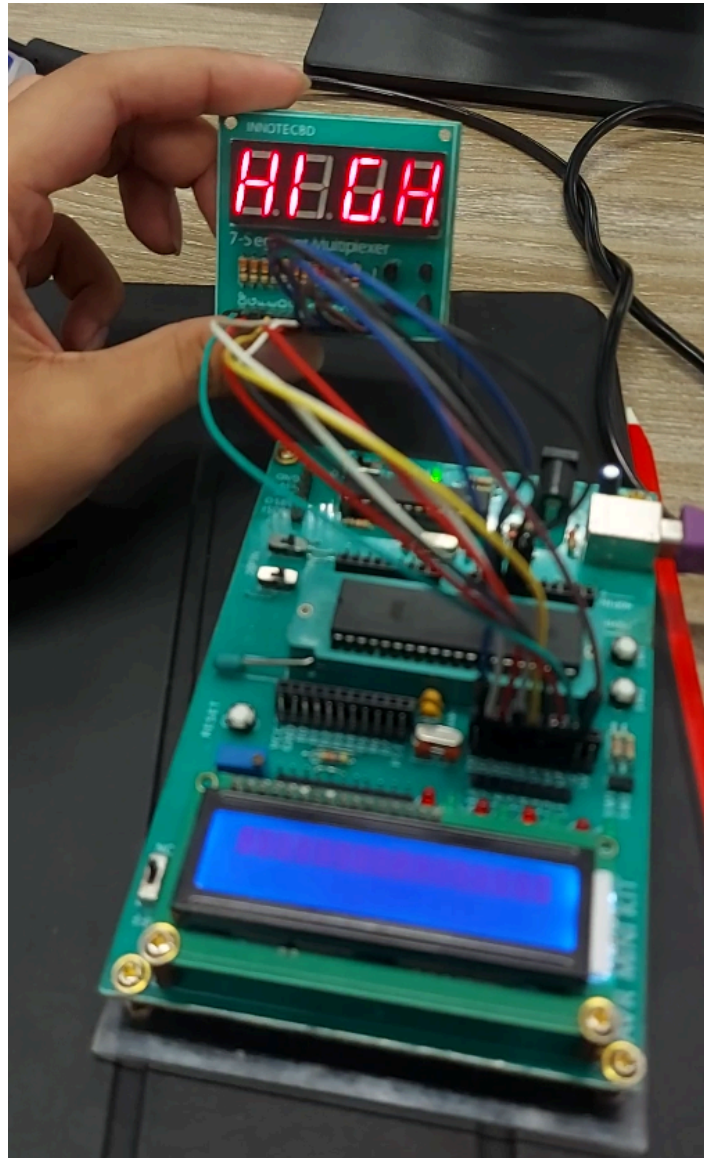


Fig 7: Displaying HIGH on 7-Segment Displays

- To Blink 'HI' on 7-Segment display

The Code

```
#include <mega32.h>
#include <delay.h>
#define data_port PORTD
#define control_port PORTC
char digit_cathode[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
char first,last,i=0,j;
void main(void)
{
    DDRD=0xFF;
    DDRC=0xC0;
    data_port=0;
    control_port=0;
while (1)
    {
        for(i=0;i<20;i++){
            control_port=0b10000000; //0b01000000; S3(C6) is for the 2nd last digit
            data_port=0x76;
            delay_ms(10);
            control_port=0b01000000;
            data_port=0x06;
            delay_ms(10);
            control_port=0b11000000;
            data_port=0x00;
        }

        delay_ms(1000);

    }
}
```

The Output:

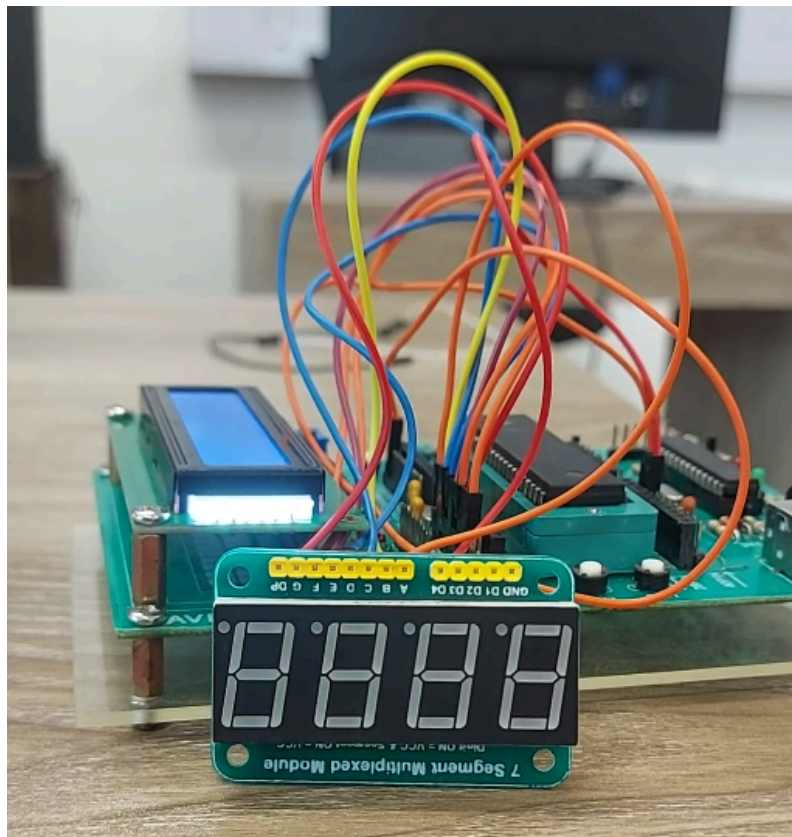


Fig 8.1: Blinking HI on 7 Segment display (7-Segment displays OFF)

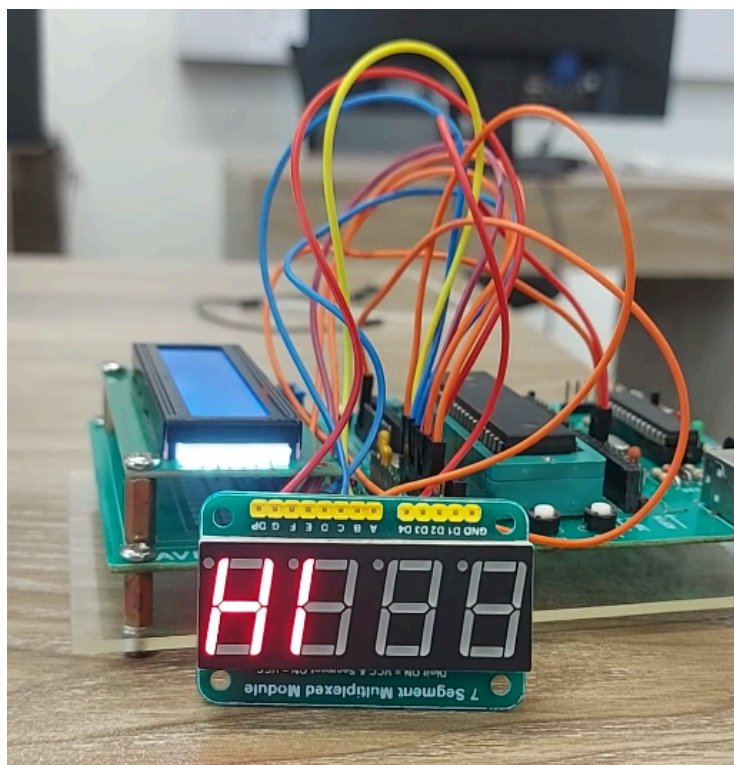


Fig 8.2: Blinking HI on 7 Segment display (7-Segment displays ON)

EEE373L

Embedded System Design Laboratory

Lab Report



Inspiring Excellence

Section:02

Group No:02

Experiment no:04

Name of the experiment: Single and Multiple External Interrupts

Prepared by:

Name: Md. Hamin-Al-Araf

ID: 21221019

Signature

All Group members:

Sl.	Name	ID	Signature
1.	Yasir Anaf Siddique	21221073	
2.	Md Talha Bin Imran	21221018	
3.	Nafim Karim Khan	21221039	
4.	Afrida Islam	21221030	

Date of Submission: 22/06/2024

Objective: The objective of this experiment is to understand the basics of external single interrupts and multiple interrupts.

Theoretical Background:

An interrupt is a mechanism that allows a microcontroller to temporarily pause its normal, sequential task execution to address a different, more urgent task. In typical operation, a microcontroller executes a program line by line. However, when a programmer needs to divert the microcontroller's attention to an immediate task, an interrupt is used. During such events, the microcontroller raises a specific flag related to the task. The microcontroller is configured to detect this flag and, using an Interrupt Service Routine (ISR), automatically handles the task whenever the flag is activated, resuming normal operation afterward.

In AVR 8-bit microcontrollers, there are both internal and external sources of interrupts. Internal interrupts are tied to the microcontroller on board peripherals, such as the Timer/Counter or the Analog Comparator. External interrupts, on the other hand, are triggered via dedicated external pins. For example, the AVR ATmega32 microcontroller supports four external interrupts:

- RESET Interrupt: Triggered by pin 9.
- External Interrupt 0 (INT0): Triggered by pin 16.
- External Interrupt 1 (INT1): Triggered by pin 17.
- External Interrupt 2 (INT2): Triggered by pin 3.

This experiment will mainly focus on external interrupts. The first part, PART A will demonstrate the use of a single interrupt and the second part, PART B, will demonstrate the use of multiple interrupts.

PART A (Single External Interrupt)

Circuit Diagram

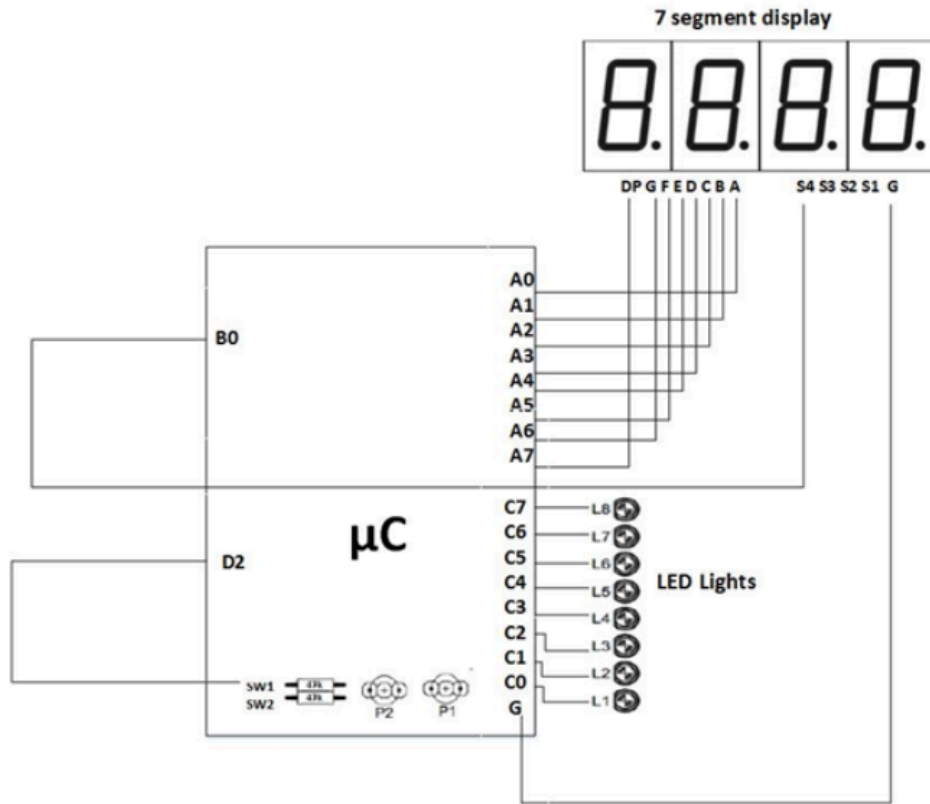


Figure 01: Circuit diagram of Single External Interrupt task

Code

```
#include <mega32.h>
#include <delay.h>

interrupt [EXT_INT0] void ext_int0_isr(void)
{
    unsigned int cathode[10]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
    unsigned int i=0,k=0;
    PORTB=0x01;
    for (i=0;i<10;i++)
    {
        PORTA= cathode[i];
        delay_ms(1000);
    }
}
```

```

    PORTA=0;
    PORTB=0;
}

void main(void)
{
    char i;
    DDRA=0xFF;
    DDRC=0xFF;
    DDRB=0x01;
    PORTA=0;
    PORTC=0;
    PORTB=0;

    GICR|=(0<<INT1) | (1<<INT0) | (0<<INT2);
    MCUCR=(0<<ISC11) | (0<<ISC10) | (1<<ISC01) | (0<<ISC00);
    MCUCSR=(0<<ISC2);
    GIFR=(0<<INTF1) | (1<<INTF0) | (0<<INTF2);

    #asm("sei")
    while (1)
    {
        PORTC=0x80;
        for (i=0;i<8;i++)
        {
            delay_ms(1000);
            PORTC>>=1;
        }
    }
}

```

Output

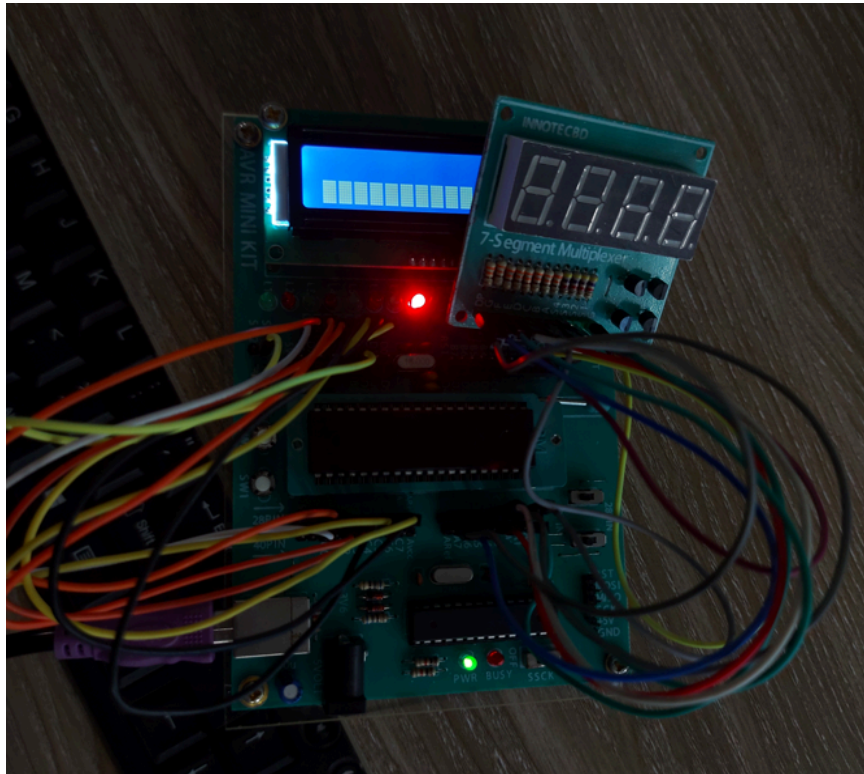


Figure 02: LED keeps shifting (Infinite While Loop task)

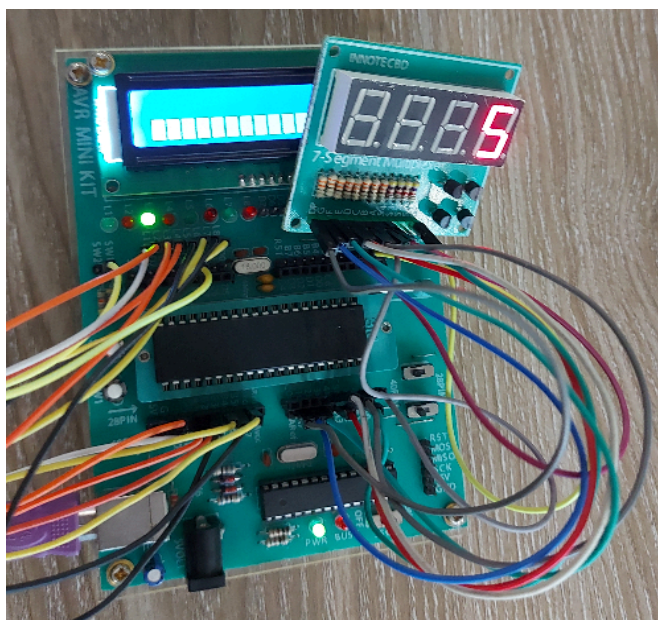


Figure 03: Interrupted Task. 7 Segment display starts showing countdown

Discussion: This task focuses on a single external interrupt. For the interrupt, PORTD.2 (INT0) is used. When the code is run, it executes the task defined in the infinite loop which is the LEDs shifting towards right. When the interrupt button is pressed, the LEDs stop shifting and the 7-segment display starts showing a countdown from 0 to 9 with a 1 second interval. This is how an infinite while loop task is interrupted to do another task using a single interrupt.

PART B (Multiple external Interrupts)

Circuit Diagram:

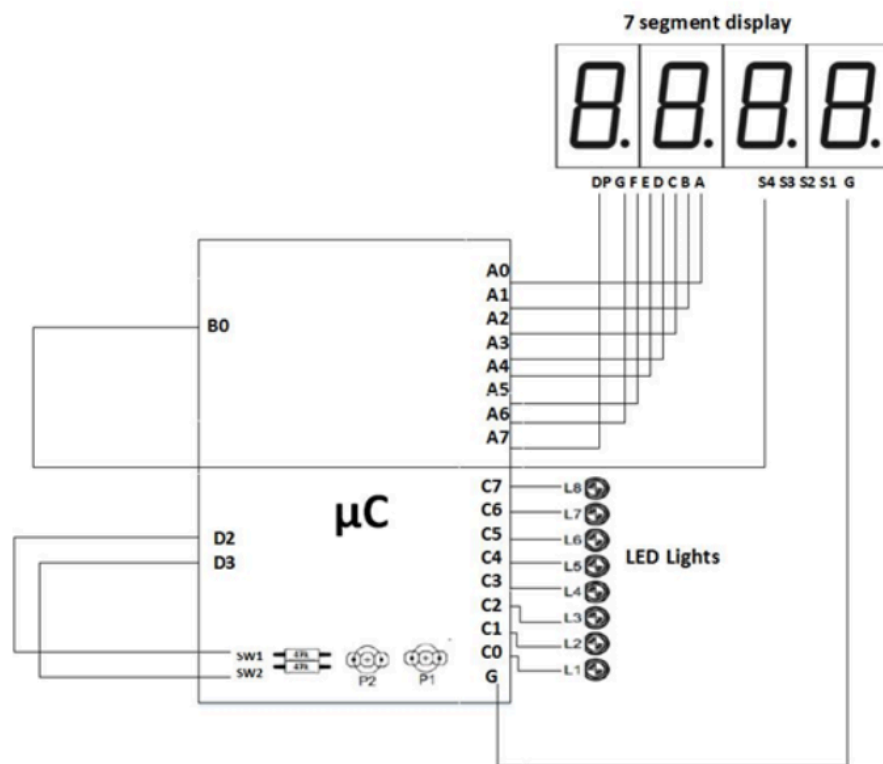


Figure 04: Circuit Diagram of Multiple External Interrupts

Code

```
#include <mega32.h>
#include <delay.h>
```



```

interrupt [EXT_INT0] void ext_int0_isr(void)
{
    unsigned int cathode[10]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
    unsigned int i=0,k=0;
    PORTB=0x01;
    for (i=0;i<10;i++)
    {
        PORTA= cathode[i];
        delay_ms(1000);
    }
    PORTA=0;
    PORTB=0;
}
interrupt [EXT_INT1] void ext_int1_isr(void)
{
    char i;
    #asm("sei")
    PORTC=0x80;
    for (i=0;i<8;i++)
    {
        delay_ms(1000);
        PORTC>>=1;
    }
}
void main(void)
{

    DDRA=0xFF;
    DDRC=0xFF;
    DDRB=0x01;
    PORTA=0;
    PORTC=0;
    PORTB=0;
    GICR|=(1<<INT1) | (1<<INT0) | (0<<INT2);
    MCUCR=(1<<ISC11) | (0<<ISC10) | (1<<ISC01) | (0<<ISC00);
    MCUCSR=(0<<ISC2);
    GIFR=(1<<INTF1) | (1<<INTF0) | (0<<INTF2);
    #asm("sei")
    while (1)
    {
        // Place your code here

    }
}

```

Output

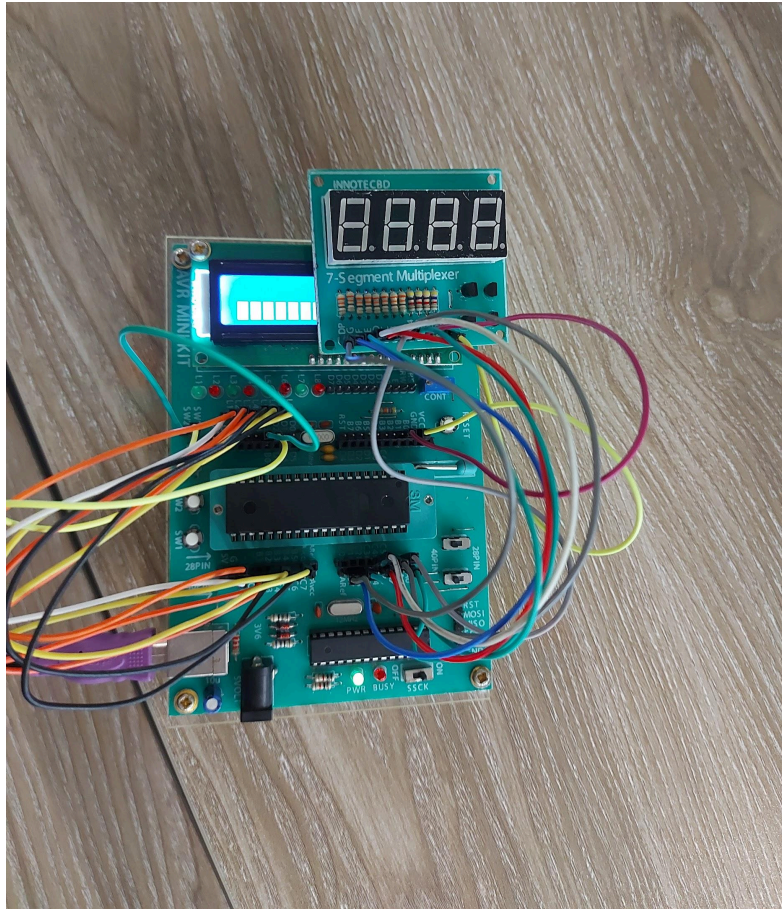


Figure 05: When no Switch is pressed, nothing happens

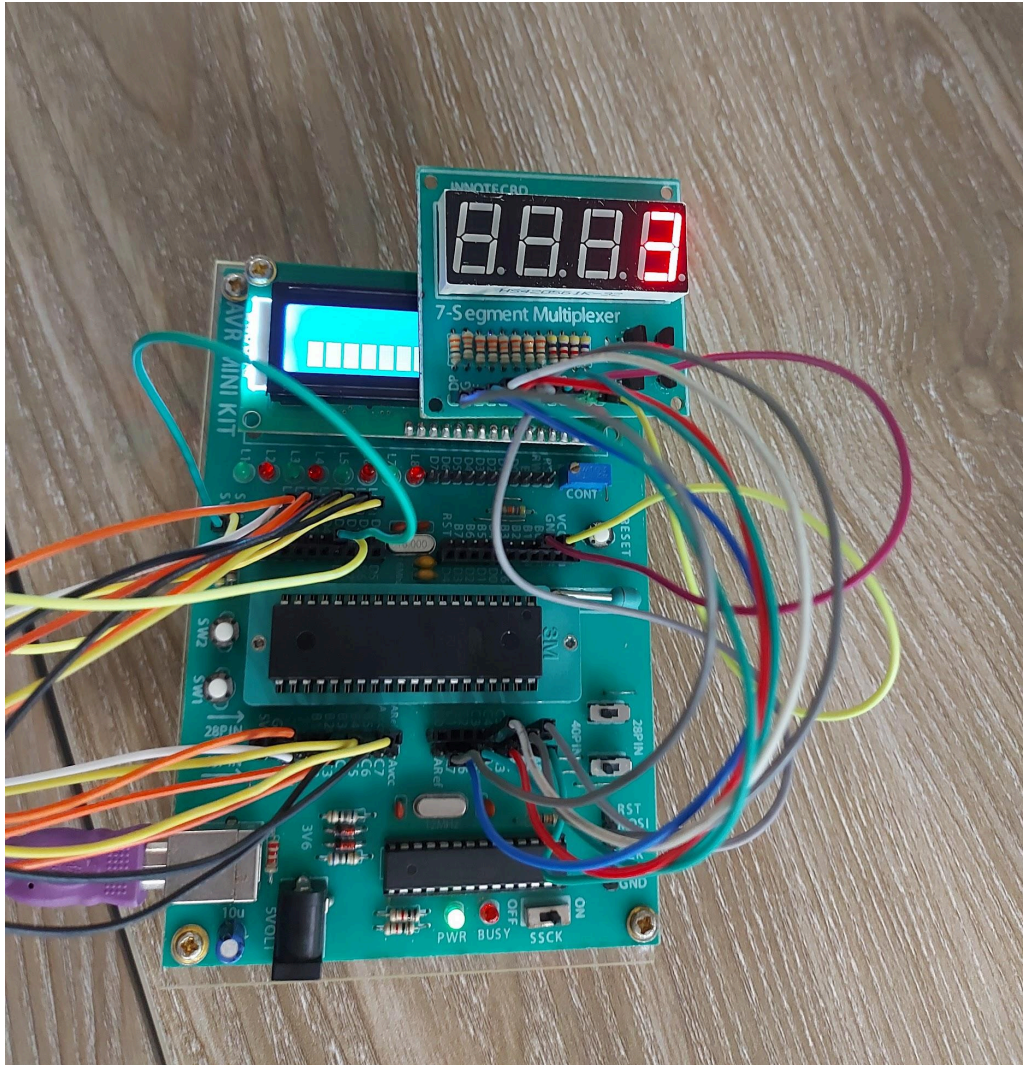


Figure 06: When Switch 1 (Priority) is pressed, 7 Segment display shows countdown from 0-9

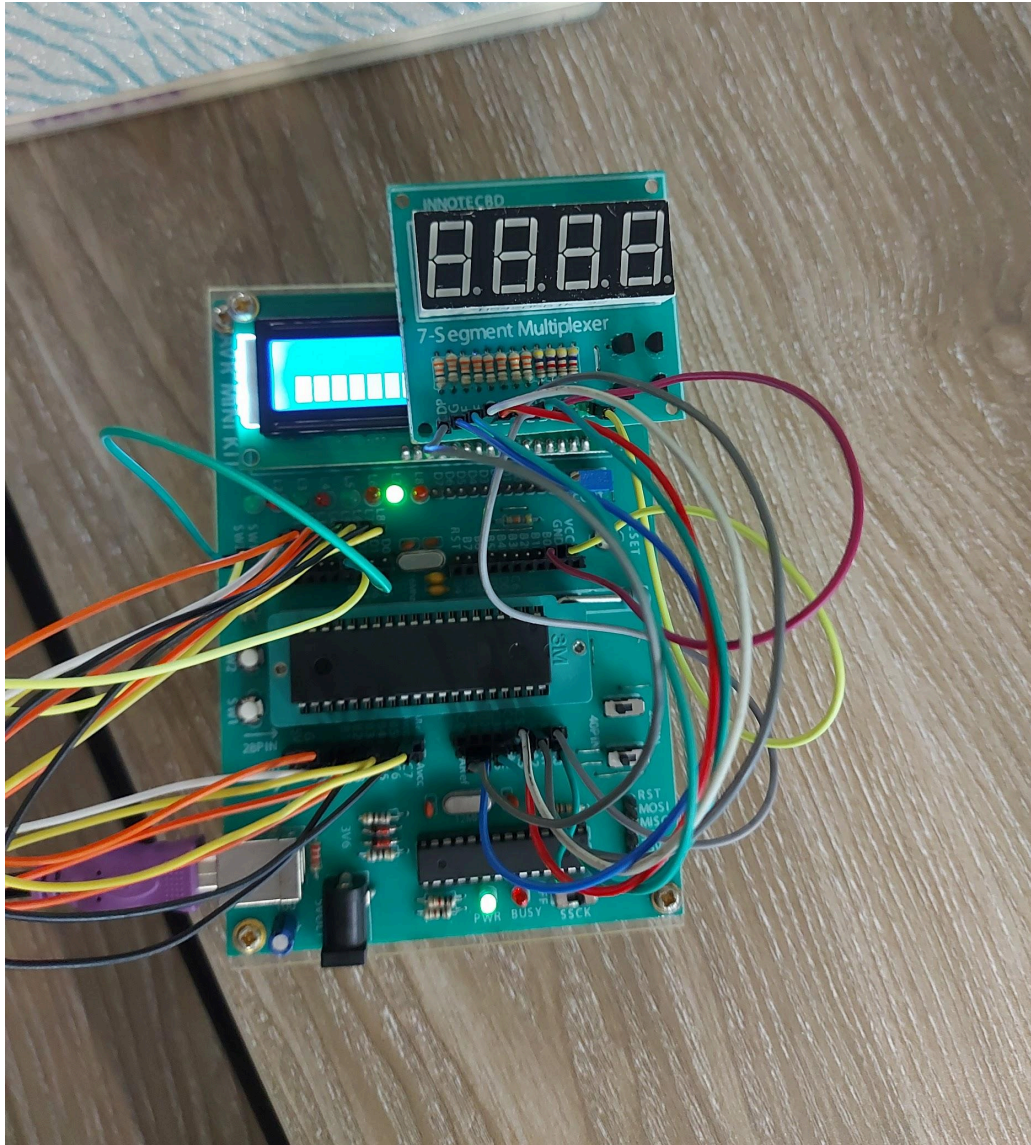


Figure 07: When Switch 2 is pressed, the led starts shifting

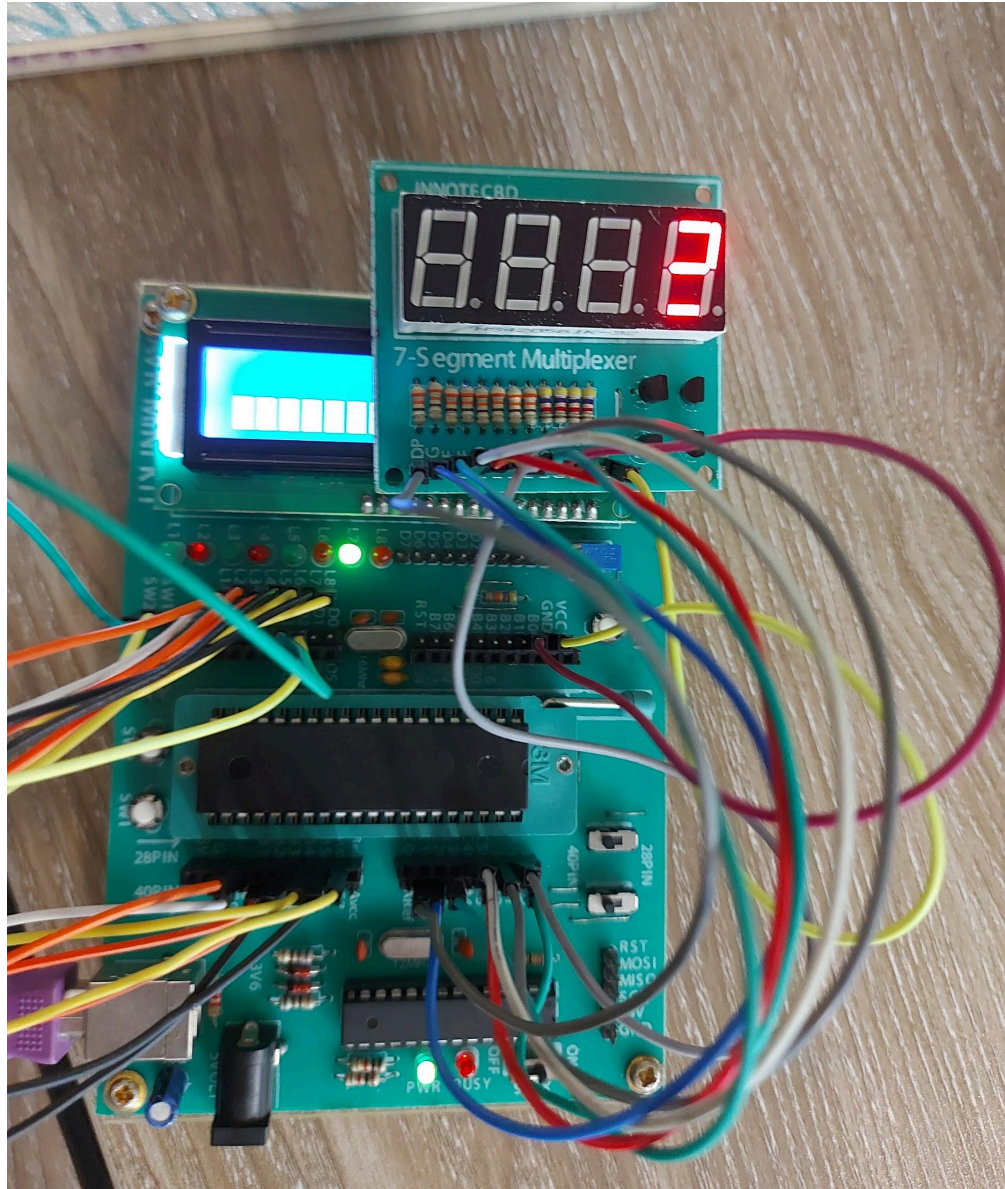


Figure 08: When Switch 1 is pressed during the LED shifting, the LED shifting is interrupted and the 7-Segment display starts.

Discussion: This task covers multiple interrupts. For the interrupts, PORTD.2 (INT0) and PORTD.3(INT1) are used. Once the code is uploaded, nothing really happens because it's waiting for one of the switches to be pressed. When Switch 1 (INT0) is pressed, the 7-Segment display shows a countdown from 0-9 with a 1 second delay in between. During this process even if Switch 2 (INT1) is pressed, the task is not interrupted since INT0 is high priority. However when Switch 2 (INT1) is pressed, the LEDs start shifting and during this task if Switch 1(INT0) is pressed, the LED shifting is interrupted and paused and the 7-Segment display starts showing the count down and when that's over, the LED shifting task is resumed. Hence INT0 (7-Segment

task) has higher priority over INT1 (LED shifting) and to set this, the global interrupt, `#asm("sei")` is declared in the INT0 task to be the priority.

Lab Report Task

Modify the code in such a way that will set INT1 higher than INT0

Code

```
#include <mega32.h>
#include <delay.h>
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    unsigned int cathode[10]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
    unsigned int i=0,k=0;
    PORTB=0x01;
    #asm("sei")
    for (i=0;i<10;i++)
    {
        PORTA= cathode[i];
        delay_ms(1000);
    }
    PORTA=0;
    PORTB=0;
}
interrupt [EXT_INT1] void ext_int1_isr(void)
{
    char i;
    PORTC=0x80;
    for (i=0;i<8;i++)
    {
        delay_ms(1000);
        PORTC>>=1;
    }
}
void main(void)
{
    DDRA=0xFF;
    DDRC=0xFF;
    DDRB=0x01;
    PORTA=0;
    PORTC=0;
    PORTB=0;
    GICR|=(1<<INT1) | (1<<INT0) | (0<<INT2);
    MCUCR=(1<<ISC11) | (0<<ISC10) | (1<<ISC01) | (0<<ISC00);
```

```
MCUCSR=(0<<ISC2);
GIFR=(1<<INTF1) | (1<<INTF0) | (0<<INTF2);
asm("sei")
while (1)
{

}
}
```

EEE373L

Embedded System Design Laboratory

Lab Report



Inspiring Excellence

Section:02

Group No:02

Experiment no:05

Name of the experiment: **Development of a Stopwatch (Use of Timer1)**

Prepared by:

Name: Yasir Anaf Siddique

ID: 21221073

Signature

All Group members:

Sl.	Name	ID	Signature
1.	Afrida Islam	21221030	
2.	Md Talha Bin Imran	21221018	
3.	Md. Hamin-Al-Araf	21221019	
4.	Nafim Karim Khan	21221039	

Date of Submission: 22/09/2024

Objective:

The objective of this experiment is to develop a stopwatch for measuring time in seconds and minutes. The stopwatch should have starting, stopping and resetting facilities.

Apparatus:

1. ATmega32
2. 7 Segment Display
3. Connecting Wires

Connection Diagram:

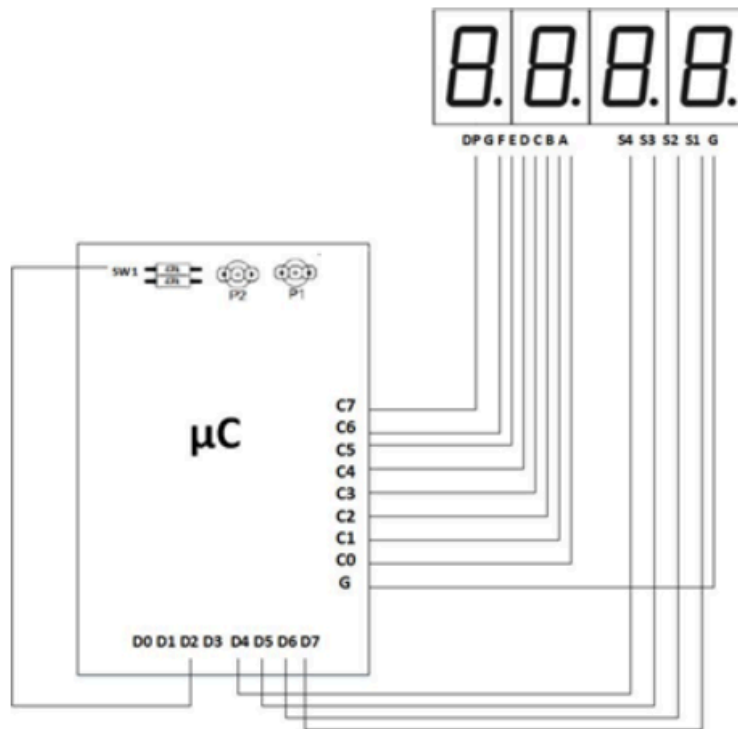


Figure-1: Circuit Diagram of a Stopwatch using ATmega32

The Code:

```
#include <mega32.h>
#include <delay.h>

// Define ports for 7-segment display
#define output_port PORTC
#define control_port PORTD

// Variables to hold the current time (seconds and minutes)
char sec = 45, min = 57, s1, s2, m1, m2, press = 0;

// Array showing the 7-segment cathode patterns for digits 0-9
char digit_cathod[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

char digit_cathod_decimal[10] = {0xBF, 0x86, 0xDB, 0xCF, 0xE6, 0xED, 0xFD, 0x87, 0xFF, 0xEF}; // | 0x80

// Function to separate the digits of seconds and minutes
void seperate_digits(void)
{
    s1 = sec % 10;
    s2 = sec / 10;
    m1 = min % 10;
    m2 = min / 10;
}

// Function to display the digits on the 7-segment display
void display_digits(void)
{
    control_port = 0b00010000;
    output_port = digit_cathod[s1];
    delay_ms(10);

    control_port = 0b00100000;
    output_port = digit_cathod[s2];
    delay_ms(10);

    control_port = 0b01000000;
    output_port = digit_cathod_decimal[m1];
    delay_ms(10);

    control_port = 0b10000000;
    output_port = digit_cathod[m2];
    delay_ms(10);
}
```

```

// Interrupt service routine for external interrupt INT0
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    press++;                // Increment the button press counter
    if (press == 3)         // Reset time after 3 presses
    {
        press = 0;
        min = 0;
        sec = 0;
    }
}

// Interrupt service routine for Timer1 Compare A
interrupt [TIM1_COMPA] void timer1_compa_isr(void)
{
    // Reset the Timer1 counter
    TCNT1H = 0x00;
    TCNT1L = 0x00;

    if (press == 1)
    {
        sec++;              // Increment seconds
        if (sec == 60)      // If seconds reach 60, reset to 0 and increment
minutes
        {
            sec = 0;
            min++;
            if (min == 60)   // If minutes reach 60, reset to 0
                min = 0;
        }
    }
}

void main(void)
{
    DDRC = 0xFF; // Set PORTC as output (for the 7-segment display)

    DDRD = 0xF0; // Set the upper 4 bits of PORTD as output (for controlling the
7-segment displays)

    // Configure Timer1 for normal mode with a 1024 prescaler
    TCCR1A = (0 << COM1A1) | (0 << COM1A0) | (0 << COM1B1) | (0 << COM1B0) | (0 <<
WGM11) | (0 << WGM10);
    TCCR1B = (0 << ICNC1) | (0 << ICES1) | (0 << WGM13) | (0 << WGM12) | (1 << CS12) |
(0 << CS11) | (1 << CS10);

    TCNT1H = 0x00;
    TCNT1L = 0x00;
}

```

```

// Set the Timer1 Compare A register to a specific value
OCR1AH = 0x3D;
OCR1AL = 0x09;

TIMSK = (0 << OCIE2) | (0 << TOIE2) | (0 << TICIE1) | (1 << OCIE1A) | (0 << OCIE1B)
| (0 << TOIE1) | (0 << OCIE0) | (0 << TOIE0); // Enable Timer1 Compare A interrupt

GICR |= (0 << INT1) | (1 << INT0) | (0 << INT2); // Enable external interrupt INT0

MCUCR = (0 << ISC11) | (0 << ISC10) | (1 << ISC01) | (0 << ISC00); // Set INT0 to
trigger on a falling edge

GIFR = (0 << INTF1) | (1 << INTF0) | (0 << INTF2); // Clear INT0 flag

#asm("sei") // Enable global interrupts

// loop to update and display the time continuously
while (1)
{
    seperate_digits();
    display_digits();
}
}

```


Outputs:

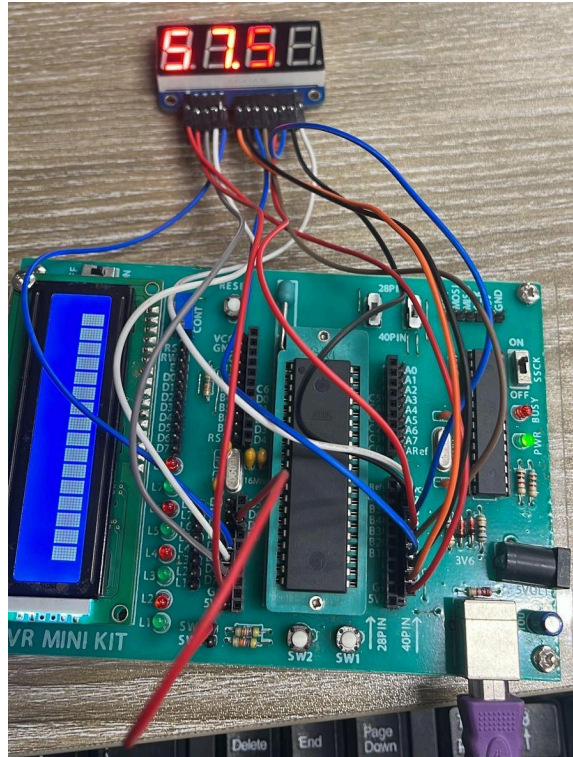


Figure-2: Timer1 digital Stopwatch in running state

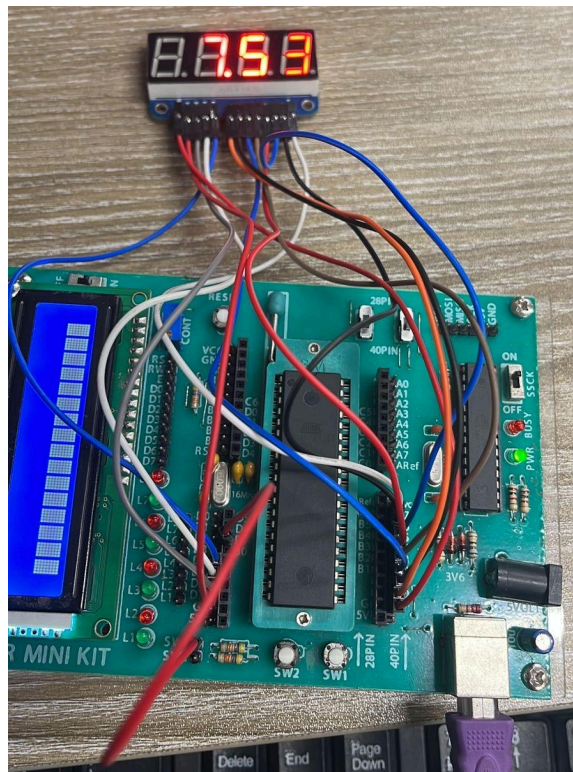


Figure-3: Timer1 digital Stopwatch in stop/ pause state

Discussion:

In this experiment, a stopwatch was implemented using the ATmega32 microcontroller, 16-bit Timer1, and 7-segment displays. Timer1 was configured with a prescaler of 1024, and in output compare match mode, it reached the value 0x3D09 to increment the time every second. The time was displayed on four 7-segment displays, showing minutes and seconds, using the functions `seperate_digits()` and `display_digits()`. External interrupts were used to pause and reset the timer, triggered by an external button press. When the button was pressed once, the time paused, and when pressed again, the timer reset to 0. Additionally, multiplexing was employed to efficiently manage the four 7-segment displays, while the Timer1 interrupt service routine ensured that the time was updated accurately every second.

Lab Report Tasks:

1. Code-

```
#include <mega32.h>
#include <delay.h>

// Define ports for 7-segment display
#define output_port PORTC
#define control_port PORTD

// Variables to hold the current time (seconds and minutes)
char sec = 45, min = 57, s1, s2, m1, m2, press = 0;
char pause = 0; // Flag to indicate pause state

// Array showing the 7-segment cathode patterns for digits 0-9
char digit_cathod[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

char digit_cathod_decimal[10] = {0xBF, 0x86, 0xDB, 0xCF, 0xE6, 0xED, 0xFD, 0x87, 0xFF, 0xEF}; // | 0x80

// Function to separate the digits of seconds and minutes
void seperate_digits(void)
{
    s1 = sec % 10;
    s2 = sec / 10;
    m1 = min % 10;
    m2 = min / 10;
}

// Function to display the digits on the 7-segment display
void display_digits(void)
{
    // If paused, blink the display
    if (pause == 1)
```

```

{
    // Turn off the display for blinking effect
    output_port = 0x00;
    delay_ms(300); // Adjust blink speed here
    return;
}

control_port = 0b00010000;
output_port = digit_cathod[s1];
delay_ms(10);

control_port = 0b00100000;
output_port = digit_cathod[s2];
delay_ms(10);

control_port = 0b01000000;
output_port = digit_cathod_decimal[m1];
delay_ms(10);

control_port = 0b10000000;
output_port = digit_cathod[m2];
delay_ms(10);
}

// Interrupt service routine for external interrupt INT0
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    press++; // Increment the button press counter
    if (press == 3) // Reset time after 3 presses
    {
        press = 0;
        min = 0;
        sec = 0;
    }
    else if (press == 1) // Enter pause state
    {
        pause = 1; // Set pause flag to enable blinking
    }
    else if (press == 2) // Exit pause state
    {
        pause = 0; // Clear pause flag to stop blinking
    }
}

// Interrupt service routine for Timer1 Compare A
interrupt [TIM1_COMPA] void timer1_compa_isr(void)
{
    // Reset the Timer1 counter

```

```

    TCNT1H = 0x00;
    TCNT1L = 0x00;

    if (press == 1)
    {
        sec++; // Increment seconds
        if (sec == 60) // If seconds reach 60, reset to 0 and increment
minutes
        {
            sec = 0;
            min++;
            if (min == 60) // If minutes reach 60, reset to 0
                min = 0;
        }
    }
}

void main(void)
{
    DDRC = 0xFF; // Set PORTC as output (for the 7-segment display)

    DDRD = 0xF0; // Set the upper 4 bits of PORTD as output (for controlling the
7-segment displays)

    // Configure Timer1 for normal mode with a 1024 prescaler
    TCCR1A = (0 << COM1A1) | (0 << COM1A0) | (0 << COM1B1) | (0 << COM1B0) | (0 <<
WGM11) | (0 << WGM10);
    TCCR1B = (0 << ICNC1) | (0 << ICES1) | (0 << WGM13) | (0 << WGM12) | (1 << CS12) |
(0 << CS11) | (1 << CS10);

    TCNT1H = 0x00;
    TCNT1L = 0x00;

    // Set the Timer1 Compare A register to a specific value
    OCR1AH = 0x3D;
    OCR1AL = 0x09;

    TIMSK = (0 << OCIE2) | (0 << TOIE2) | (0 << TICIE1) | (1 << OCIE1A) | (0 << OCIE1B)
| (0 << TOIE1) | (0 << OCIE0) | (0 << TOIE0); // Enable Timer1 Compare A interrupt

    GICR |= (0 << INT1) | (1 << INT0) | (0 << INT2); // Enable external interrupt INT0

    MCUCR = (0 << ISC11) | (0 << ISC10) | (1 << ISC01) | (0 << ISC00); // Set INT0 to
trigger on a falling edge

    GIFR = (0 << INTF1) | (1 << INTF0) | (0 << INTF2); // Clear INT0 flag

    #asm("sei") // Enable global interrupts

```



```

// loop to update and display the time continuously
while (1)
{
    seperate_digits();
    display_digits();
}
}

```

Discussion: In the updated code, a pause flag is added which will be used to track when the stopwatch is paused. This detection will allow the display to blink during this pause state. When the stopwatch is paused, the display_digits() function checks whether this flag is set. If detected, it turns off and turns on the 7-segment display, creating the blink effect.

2. Code-

```

#include <mega32.h>
#include <delay.h>

#define output_port PORTC
#define control_port PORTD

char sec = 45, min = 57, hour = 1; // Added hour variable
char s1, s2, m1, m2, h1, h2, press = 0;
char digit_cathod[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
char digit_cathod_decimal[10] = {0xBF, 0x86, 0xDB, 0xCF, 0xE6, 0xED, 0xFD, 0x87, 0xFF, 0xEF};

void separete_digits(void)
{
    s1 = sec % 10;
    s2 = sec / 10;
    m1 = min % 10;
    m2 = min / 10;
    h1 = hour % 10; // Separate hour digits
    h2 = hour / 10;
}

void display_digits(void)
{
    control_port = 0b00000001; // Display first digit of hours

```

```

output_port = digit_cathod[h2];
delay_ms(10);

control_port = 0b00000010; // Display second digit of hours
output_port = digit_cathod[h1];
delay_ms(10);

control_port = 0b00000100; // Display first digit of minutes
output_port = digit_cathod[m2];
delay_ms(10);

control_port = 0b00001000; // Display second digit of minutes
output_port = digit_cathod_decimal[m1]; // With decimal for clarity
delay_ms(10);

control_port = 0b00010000; // Display first digit of seconds
output_port = digit_cathod[s2];
delay_ms(10);

control_port = 0b00100000; // Display second digit of seconds
output_port = digit_cathod[s1];
delay_ms(10);
}

interrupt [EXT_INT0] void ext_int0_isr(void)
{
    press++;
    if (press == 3)
    {
        press = 0;
        hour = 0;
        min = 0;
        sec = 0;
    }
}

interrupt [TIM1_COMPA] void timer1_compa_isr(void)
{
    TCNT1H = 0x00;
    TCNT1L = 0x00;
    if (press == 1)
    {
        sec++;
        if (sec == 60)
        {
            sec = 0;
            min++;
            if (min == 60)

```

```

        {
            min = 0;
            hour++; // Increment hour when minute reaches 60
            if (hour == 24) hour = 0; // Reset hours after 24
        }
    }
}

void main(void)
{
    DDRC = 0xFF; // Set PORTC as output for 7-segment display
    DDRD = 0x3F; // Set PORTD as output for control signals

    TCCR1A = (0 << COM1A1) | (0 << COM1A0) | (0 << COM1B1) | (0 << COM1B0) | (0 <<
WGM11) | (0 << WGM10);
    TCCR1B = (0 << ICNC1) | (0 << ICES1) | (0 << WGM13) | (0 << WGM12) | (1 << CS12) |
(0 << CS11) | (1 << CS10);
    TCNT1H = 0x00;
    TCNT1L = 0x00;
    ICR1H = 0x00;
    ICR1L = 0x00;
    OCR1AH = 0x3D;
    OCR1AL = 0x09;
    OCR1BH = 0x00;
    OCR1BL = 0x00;

    TIMSK = (0 << OCIE2) | (0 << TOIE2) | (0 << TICIE1) | (1 << OCIE1A) | (0 << OCIE1B)
| (0 << TOIE1) | (0 << OCIE0) | (0 << TOIE0);

    GICR |= (0 << INT1) | (1 << INT0) | (0 << INT2);
    MCUCR = (0 << ISC11) | (0 << ISC10) | (1 << ISC01) | (0 << ISC00);
    MCUCSR = (0 << ISC2);
    GIFR = (0 << INTF1) | (1 << INTF0) | (0 << INTF2);

    #asm("sei") // Enable global interrupts

    while (1)
    {
        separte_digits(); // Update digit separation
        display_digits(); // Display hour, minute, and second
    }
}

```

Discussion:

In the updated code, the stopwatch now tracks hours in addition to minutes and seconds. An hour variable is added and the `seperate_digits()` function to break the hours into two digits for display on the 7-segment. The `display_digits()` function now shows hours, minutes, and seconds.

The Timer1 interrupt was adjusted to increment the hour when minutes reach 60, and the hour resets to 0 after 24, keeping it in a 24-hour format.